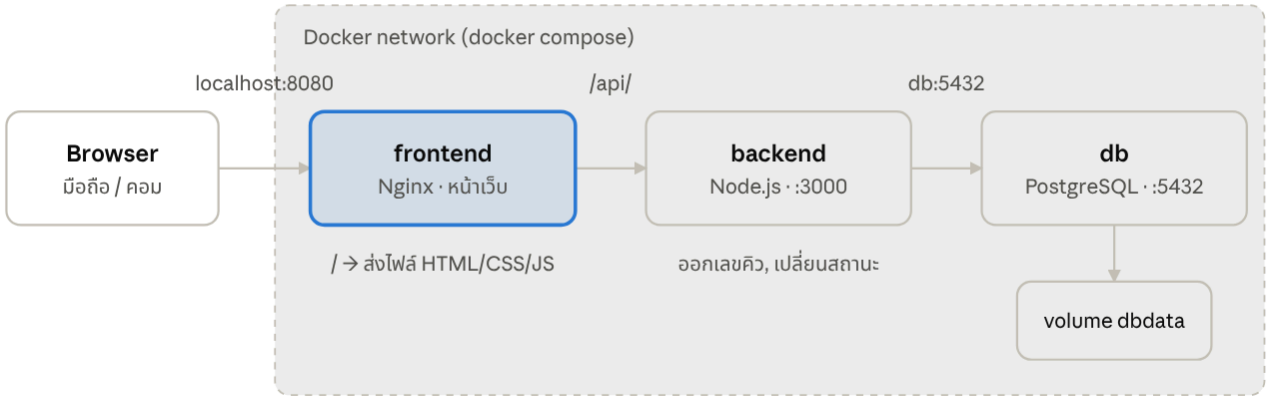
Coffee Queue: คู่มือเว็บแอป 3 Container และการ deploy บน AWS

ระบบสั่งเครื่องดื่มและติดตามคิวร้านกาแฟ แยกเป็น 3 container คือ **Nginx** (หน้าเว็บ), **Node.js** (Backend) และ **PostgreSQL** (Database) รันในเครื่องด้วย Docker Compose ก่อน แล้วขึ้น Kubernetes (k3s) บน AWS EC2 ด้วย Ansible ใช้คู่กับโค้ดใน `coffee-queue.zip`

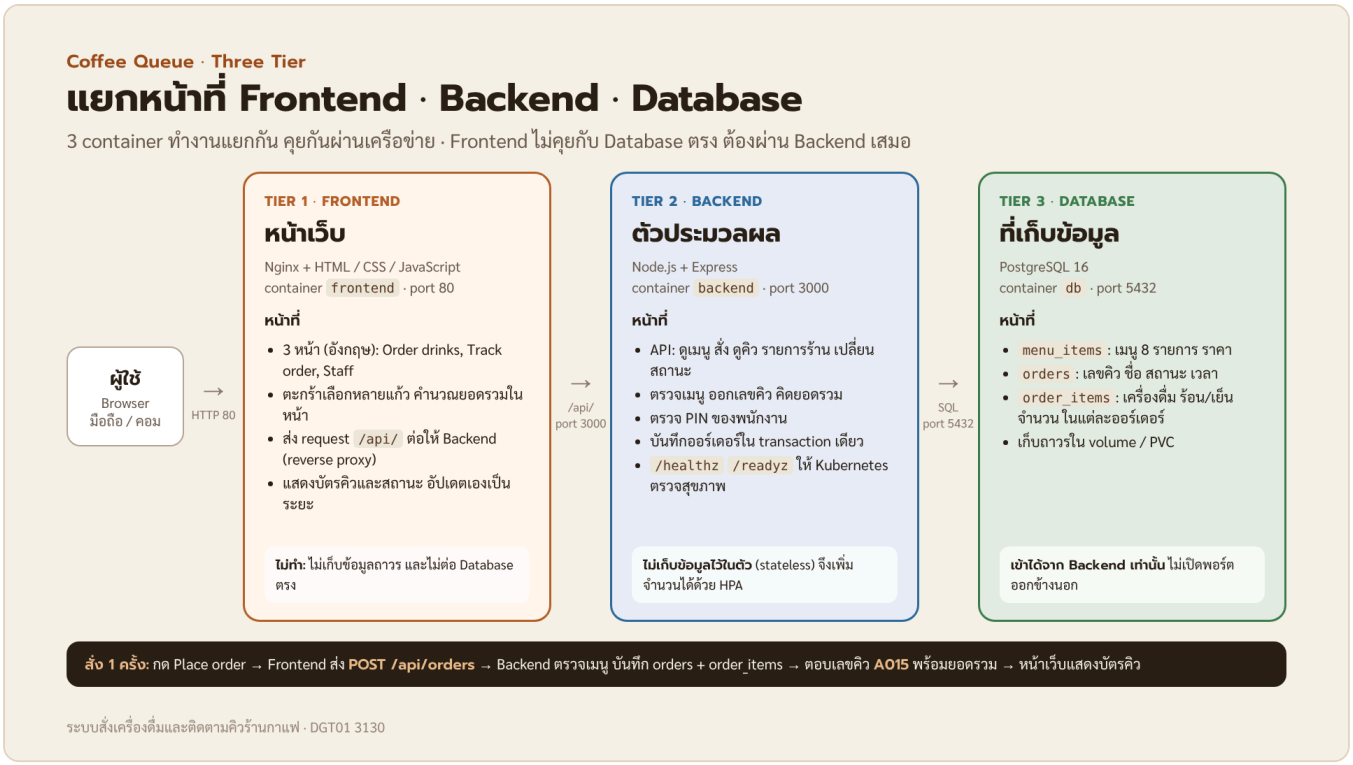
ส่วน	เนื้อหา
1–7	เว็บแอปและการเชื่อม 3 container ในเครื่อง (Docker Compose)
8	ขั้นถัดไป: deploy บน AWS EC2 ด้วย Ansible + k3s (รวมวิธีสร้าง Key Pair ใหม่)
9	ภาคผนวก: โค้ดทุกไฟล์แบบเต็ม พร้อมคำอธิบาย

1. ภาพรวม

ทุก request เข้าทาง Nginx ส่วน Backend กับ DB อยู่ข้างใน



Browser เข้าได้ทางเดียวคือ Nginx ที่ `localhost:8080` Nginx ส่งหน้าเว็บให้เอง และส่ง request ที่ขึ้นต้นด้วย `/api/` ต่อให้ Backend จากนั้น Backend ค่อยอ่านและเขียน PostgreSQL หน้าเว็บจึงไม่เคยคุยกับ Database ตรง





2. โครงสร้างไฟล์

แต่ละ container มีโฟลเดอร์ของตัวเอง ส่วน Database ใช้ image สำเร็จรูป จึงไม่มีโฟลเดอร์

```
coffee-queue/
├── README.md                  สรุปคำสั่งแบบย่อ
├── app/                       ส่วนที่ 1–7: ตัวเว็บแอป
│   ├── docker-compose.yml     สั่งรันทั้ง 3 container พร้อมกัน
│   ├── frontend/              Container 1: Nginx + หน้าเว็บ
│   │   ├── Dockerfile
```

```
│   │   ├── nginx.conf          ตั้งค่าให้ส่ง /api/ ต่อไป Backend
│   │   └── html/
│   │       ├── index.html      หน้า Order drinks (ตะกร้าหลายแก้ว)
│   │       ├── queue.html      หน้า Track order (บัตรคิว + timeline)
│   │       ├── staff.html      หน้า Staff (ใส่ PIN, แท็บตามสถานะ)
│   │       ├── app.js          ตั้งค่า API และตัวช่วยที่ทุกหน้าใช้ร่วมกัน
│   │       └── style.css
│   └── backend/                Container 2: Node.js API
│       ├── Dockerfile
│       ├── package.json
│       └── server.js
├── scripts/                    ส่วนที่ 8: build/push image, เซ็ก Load Balance
├── ansible/                    ส่วนที่ 8: สร้าง EC2, ติดตั้ง k3s, deploy, ลบ
├── k8s/                        ส่วนที่ 8: ไฟล์ Kubernetes 7 ไฟล์
├── load-test/                  ส่วนที่ 8: สคริปต์ k6 จำลองคนสั่งพร้อมกัน
└── docs/                       คู่มือนี้ + ตารางผลทดสอบ + AI usage log
```

3. แต่ละ container ทำอะไร

มีแค่ frontend ที่เปิดพอร์ตออกมาให้เครื่องเราเข้า อีก 2 ตัวคุยกันแค่ในเครือข่ายของ Docker

Container	หน้าที่	Image	พอร์ต
frontend	Nginx ส่งหน้าเว็บ (HTML/CSS/JS) ให้ browser และส่ง request `/api/` ต่อไป backend	build จาก `app/frontend/Dockerfile` (ฐาน `nginx:1.27-alpine`)	80 ใน container, เปิดออกเป็น 8080
backend	Node.js + Express รับ API, ตรวจเมนู, ออกเลขคิว, คิดยอดรวม, ตรวจ PIN, เปลี่ยนสถานะ, อ่าน/เขียนฐานข้อมูล	build จาก `app/backend/Dockerfile` (ฐาน `node:20-alpine`)	3000 (ใช้ภายใน)
db	PostgreSQL เก็บ 3 ตาราง: `menu_items` (เมนู), `orders` (ออร์เดอร์), `order_items` (เครื่องดื่มในแต่ละออร์เดอร์)	`postgres:16` (สำเร็จรูป)	5432 (ใช้ภายใน)

API ที่ backend มี

Method	Path	ใช้ทำอะไร
GET	`/api/menu`	ดูเมนูและราคา
POST	`/api/orders`	สั่งเครื่องดื่ม ส่ง `{ name, items: [{ drink, temp, quantity }] }` ได้เลขคิวและยอดรวม
GET	`/api/orders/A015`	ดูสถานะคิว (ใช้ได้ทั้ง `A015` และ `15`)
GET	`/api/staff/orders`	ออร์เดอร์ล่าสุดสำหรับหน้าร้าน (ต้องส่ง PIN ใน header `X-Staff-PIN`)

Method	Path	ใช้ทำอะไร
PATCH	`/api/orders/A015/status`	เปลี่ยนสถานะ ส่ง `{ status: "making" }` (ต้องมี PIN)
GET	`/healthz`, `/readyz`	ตรวจสุขภาพแอป และการเชื่อม DB (ให้ Kubernetes ใช้)

สถานะมี 4 ค่า: `waiting` (In queue) → `making` (Being made) → `ready` (Ready for pickup) → `done` (Picked up) หน้าเว็บและข้อความ error จาก Backend เป็นภาษาอังกฤษทั้งหมด

Backend สร้างตารางและใส่เมนูตัวอย่าง 8 รายการให้เองตอนเริ่มระบบ ไม่ต้องเตรียม SQL เอง และใส่ชื่อ container ไว้ใน header `X-Served-By` ทุก response เพื่อใช้พิสูจน์ Load Balance ตอนขึ้น Kubernetes

4. เชื่อม 3 container ยังไง

การเชื่อมมี 2 จุด: Nginx ส่งต่อไป Backend และ Backend ต่อไป Database ทั้งสองจุดใช้**ชื่อ service** ใน `docker-compose.yml` แทน IP เพราะ Docker ตั้งชื่อในเครือข่ายให้อัตโนมัติ

4.1 Nginx → Backend (ไฟล์ `app/frontend/nginx.conf`)

```
server {
    listen 80;

    # หน้าเว็บ (HTML/CSS/JS)
    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    # ทุก request ที่ขึ้นต้นด้วย /api ส่งต่อไป Backend container
    location /api/ {
        proxy_pass http://backend:3000;
    }
}
```

- เปิด `/` หรือ `/queue.html` Nginx ส่งไฟล์หน้าเว็บให้เอง
- หน้าเว็บเรียก `/api/...` Nginx ส่งต่อไป `http://backend:3000` ซึ่ง `backend` คือชื่อ service ใน compose
- ผลคือ browser คุยกับ Nginx ที่เดียว ไม่ต้องรู้ว่า Backend อยู่ที่ไหน และไม่มีปัญหา CORS

4.2 Backend → Database (ไฟล์ `app/docker-compose.yml`)

Backend อ่านค่าเชื่อมต่อจาก environment variable ไม่ได้เขียนไว้ในโค้ด `PGHOST: db` คือชื่อ service ของ Database

```yaml
services:
  frontend:                # Container 1: Nginx + หน้าเว็บ
    build: ./frontend
    ports:
      - "8080:80"          # เปิดออกมาที่ localhost:8080
    depends_on:
      - backend
```

```
  backend:                  # Container 2: Node.js API
    build: ./backend
    environment:
      PGHOST: db            # ชื่อ service ของ Database
      PGUSER: coffee
      PGPASSWORD: coffee123
      PGDATABASE: coffee
      STAFF_PIN: "1234"
    depends_on:
      - db

  db:                       # Container 3: PostgreSQL
    image: postgres:16
    environment:
      POSTGRES_USER: coffee
      POSTGRES_PASSWORD: coffee123
      POSTGRES_DB: coffee
    volumes:
      - dbdata:/var/lib/postgresql/data   # ข้อมูลไม่หายเมื่อลบ container

volumes:
  dbdata:
```

- มีแค่ frontend ที่มี `ports` อีก 2 ตัวเข้าได้จากในเครือข่าย Docker เท่านั้น
- Database อาจเปิดช้ากว่า Backend จึงให้ Backend ลองเชื่อมใหม่ทุก 2 วินาทีจนสำเร็จ
- รหัสผ่านในไฟล์นี้ใช้แค่ทดสอบในเครื่อง ตอนขึ้น Kubernetes จะย้ายไปเก็บใน Secret

5. วิธีรันด้วย Docker Desktop

1. ติดตั้ง **Docker Desktop** (docker.com) แล้วเปิดโปรแกรมทิ้งไว้ รอจนขึ้นว่า **Engine running**
2. แตกไฟล์ `coffee-queue.zip` แล้วเปิด Terminal (Windows ใช้ PowerShell) ในโฟลเดอร์ `coffee-queue/app`
3. รันคำสั่ง

   ```
   docker compose up --build
   ```

   ครั้งแรกจะใช้เวลาสักพัก เพราะต้องโหลด image และ build รอจนเห็นบรรทัด `Database พร้อมแล้ว`
4. เปิด browser ไปที่ `http://localhost:8080`
   - หน้าเมนู: `http://localhost:8080/`
   - หน้า Track order: `http://localhost:8080/queue.html`
   - หน้าร้าน: `http://localhost:8080/staff.html` (PIN `1234`)
5. เปิด Terminal อีกหน้าต่าง รัน `docker compose ps` ต้องเห็น 3 container สถานะ running

คำสั่งอื่นที่ใช้บ่อย

คำสั่ง	ทำอะไร
`docker compose ps`	ดูว่า container ไหนทำงานอยู่
`docker compose logs backend`	ดู log ของ Backend

คำสั่ง	ทำอะไร
`docker compose down`	หยุดระบบ (ข้อมูลยังอยู่)
`docker compose down -v`	หยุดระบบและลบข้อมูลทั้งหมด
`docker compose up --build`	รันใหม่หลังแก้โค้ด

6. ทดสอบว่าเชื่อมกันครบ

ทำตามลำดับ ถ้าผ่านทุกข้อแปลว่า 3 container คุยกันได้ครบ บันทึกผลจริงลงในคอลัมน์สุดท้ายเป็นหลักฐาน

#	ทดสอบ	พิสูจน์ว่า	ผลที่ควรได้	ผลจริง
1	เปิด `localhost:8080`	Nginx ทำงาน	เห็นหน้า Order drinks ของ Coffee Queue	
2	ดูรายการเครื่องดื่ม	Nginx → Backend → DB	มีเมนู 8 รายการพร้อมราคา	
3	ใส่ชื่อ เลือก 2 รายการ กด Add to order แล้ว Place order	เขียนลง DB ได้	ได้บัตรคิว เช่น A001 พร้อมยอดรวม	
4	กด Track my order	อ่านจาก DB ได้	สถานะ In queue	
5	เปิด `staff.html` ใส่ PIN `1234` กด Start making	อัปเดต DB ได้	ออร์เดอร์ย้ายไปแท็บ Being made และหน้า Track order เปลี่ยนเองภายใน 8 วินาที	
6	ใส่ PIN ผิด	การตรวจ PIN ทำงาน	ขึ้น Wrong PIN. Please try again.	
7	`docker compose down` แล้ว `up` ใหม่ เช็กคิวเดิม	ข้อมูลอยู่ใน volume	คิวเดิมยังอยู่	

ตรวจ API ตรงผ่าน Nginx ก็ได้ (ต้องได้รายการเมนูเป็น JSON และ header `X-Served-By` เป็นชื่อ container ของ Backend):

```
curl -i http://localhost:8080/api/menu
```

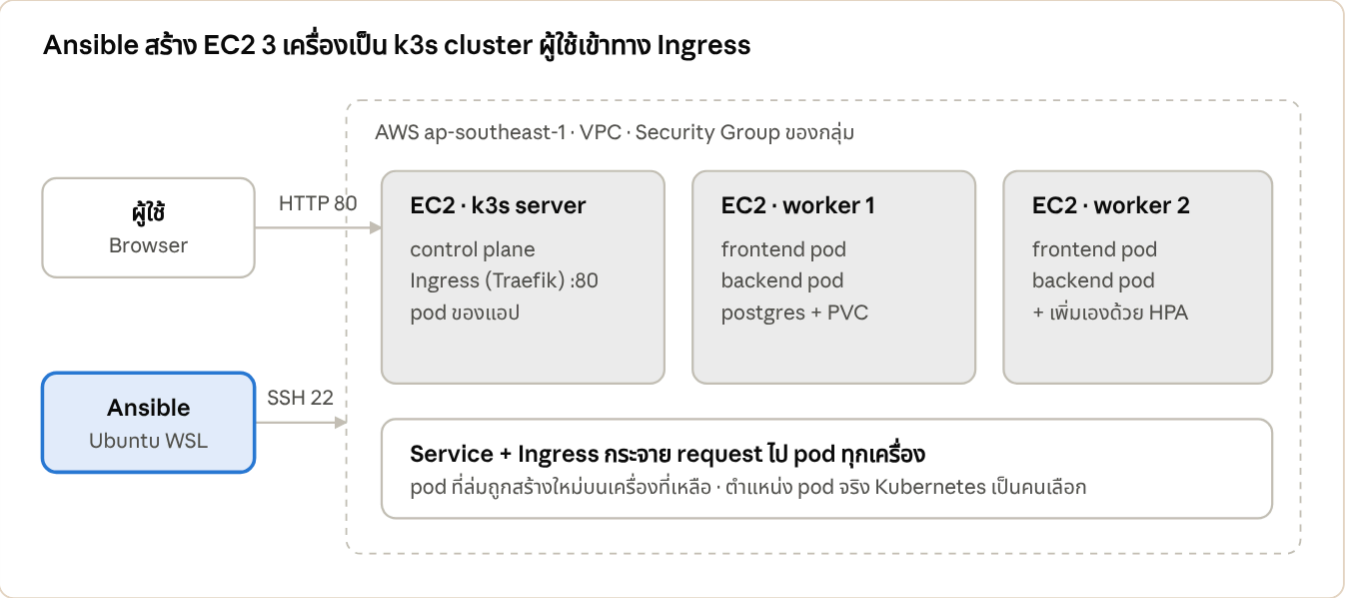
7. ปัญหาที่พบบ่อย (ในเครื่อง)

อาการ	สาเหตุ	วิธีแก้
`Cannot connect to the Docker daemon`	ยังไม่ได้เปิด Docker Desktop	เปิดโปรแกรมแล้วรอให้ขึ้น Engine running

อาการ	สาเหตุ	วิธีแก้
`port is already allocated`	มีโปรแกรมอื่นใช้พอร์ต 8080	แก้ `"8080:80"` เป็น `"8081:80"` แล้วเปิด `localhost:8081`
เปิดเว็บได้ แต่เมนูไม่ขึ้น	Backend ยังรอ Database อยู่	รอ 10–20 วินาทีแล้วรีเฟรช ดู `docker compose logs backend`
เปิดไฟล์ HTML ตรงๆ แล้วกดสั่งไม่ได้	ไม่ได้ผ่าน Nginx จึงเรียก `/api/` ไม่ได้	เปิดผ่าน `localhost:8080` เท่านั้น
สั่งแล้วขึ้น Something went wrong หรือ Unknown drink	เคยรันเวอร์ชันก่อน ตารางเก่ายังอยู่ใน volume	`docker compose down -v` แล้ว `up --build` ใหม่
แก้โค้ดแล้วหน้าเว็บไม่เปลี่ยน	image เก่ายังใช้อยู่	รัน `docker compose up --build` ใหม่
อยากเริ่มข้อมูลใหม่หมด	ข้อมูลเก่าอยู่ใน volume	`docker compose down -v` แล้ว `up` ใหม่

8. ขั้นถัดไป: deploy บน AWS EC2 ด้วย Ansible + k3s

หลัง 3 container รันผ่าน Docker Compose ได้แล้ว เราจะย้ายระบบเดียวกันขึ้น **EC2 3 เครื่อง** ที่ต่อกันเป็น Kubernetes cluster (k3s) โดยใช้ Ansible สร้างและตั้งค่าทั้งหมด รูปแบบเดียวกับ Lab 2 (provision → configure → destroy) แต่เปลี่ยนจาก Nginx LB กับ Web Server ที่ตั้งเอง มาเป็น Kubernetes ที่ scale และกู้ตัวเองได้อัตโนมัติ



8.1 ต่อยอดจาก Lab 1 และ Lab 2

เรื่อง	Lab 1	Lab 2	โปรเจกต์
เครื่อง	EC2 1 เครื่อง สร้างเองใน Console	EC2 3 เครื่อง สร้างด้วย Ansible	EC2 3 เครื่อง สร้างด้วย Ansible (k3s server 1 + worker 2)
ตัวแอป	หน้าเว็บ static	Portal static บน Web 2 เครื่อง	เว็บแอป 3 Tier เป็น container
Gateway / LB	Nginx เสิร์ฟเว็บ	Nginx กระจายไป Web 2 เครื่อง	Ingress (Traefik) + Service กระจายไปหลาย pod ข้ามเครื่อง
Scaling	ไม่มี	แก้ `web_count` แล้วรัน Ansible ใหม่ (manual)	HPA เพิ่ม Backend 2–5 pod ตาม CPU (อัตโนมัติ)
เครื่องล่ม	เว็บดับ	เหลือ Web อีกเครื่อง	pod สร้างใหม่เอง, DB เก็บใน PVC
รันซ้ำ	-	ไม่สร้างเครื่องเกิน (Instance ID เดิม)	เหมือน Lab 2 และ deploy ซ้ำแล้วได้ผลเดิม
ปิดงาน	terminate เอง	`destroy.yml`	`destroy.yml` ลบ EC2 และ Security Group ของโปรเจกต์

8.2 ไฟล์ที่ใช้ในส่วนนี้

```
scripts/
├── build-push.sh          build image แบบ x86 แล้ว push ขึ้น Docker Hub
└── check-lb.sh            ยิง 10 request ดูว่า pod ไหนตอบ
ansible/
├── ansible.cfg            ไม่ต้องตอบ yes ตอน SSH เครื่องใหม่
├── requirements.yml       collection amazon.aws
├── vars.yml               ค่าบัญชี AWS + Docker Hub  ← ต้องแก้
├── secrets.example.yml    แม่แบบรหัสผ่าน (คัดลอกเป็น secrets.yml) ← ต้องทำ
├── provision.yml          สร้าง Security Group + EC2 แล้วเขียน inventory.ini
├── k3s.yml                ติดตั้ง k3s server แล้วให้ worker join
├── deploy.yml             สร้าง Secret แล้ว apply ไฟล์ใน k8s/
├── destroy.yml            ลบ EC2 และ Security Group ของโปรเจกต์
└── templates/inventory.ini.j2
k8s/
├── 00-namespace.yaml      namespace coffee-queue
├── 01-config.yaml         ConfigMap (PGHOST, PGUSER, PGDATABASE)
├── 02-postgres.yaml       StatefulSet + PVC 1Gi + Service
├── 03-backend.yaml        Deployment 2 replicas + probes + Service
├── 04-frontend.yaml       Deployment 2 replicas + Service
├── 05-ingress.yaml        / → frontend, /api → backend
└── 06-hpa.yaml            backend 2–5 pods ที่ CPU 60%
load-test/order-load.js    k6 จำลอง 50 คนสั่งพร้อมกัน
```

8.3 เตรียมเครื่องควบคุม (Ubuntu WSL แบบ Lab 2)

ใน PowerShell พิมพ์ `wsl -d Ubuntu-24.04` แล้วรันใน Ubuntu (แก้ `<ชื่อผู้ใช้ Windows>` ให้ตรง)

```bash
# คัดลอกโปรเจกต์เข้า WSL
cp -r "/mnt/c/Users/<ชื่อผู้ใช้ Windows>/Downloads/coffee-queue" ~/
cd ~/coffee-queue/ansible
chmod +x ../scripts/*.sh

# ติดตั้ง Ansible และ Boto3 (ทำครั้งเดียว)
sudo apt update && sudo apt install -y python3-venv python3-pip curl unzip openssh-client
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install ansible-core 'boto3[crt]'
ansible-galaxy collection install -r requirements.yml

# ลงชื่อเข้า AWS (ติดตั้ง AWS CLI v2 ตาม Lab 2 ก่อน)
aws login --profile lab2 --region ap-southeast-1 --remote
```

ทำทุกครั้งที่เปิด Terminal ใหม่

```bash
cd ~/coffee-queue/ansible
source .venv/bin/activate
export AWS_PROFILE=lab2
export AWS_DEFAULT_REGION=ap-southeast-1
aws sts get-caller-identity        # ต้องแสดง Account ของเรา
```

8.4 เตรียม Key Pair (กุญแจ SSH เข้า EC2)

Key Pair ผูกกับ**บัญชี AWS** และ **Region** ถ้าใช้บัญชีเดิมที่ทำ Lab 2 และยังมีไฟล์ `.pem` ใช้แบบ A ได้ ถ้าใช้บัญชี AWS คนละอีเมล หรือหาไฟล์ `.pem` ไม่เจอ ให้สร้างใหม่แบบ B

แบบ A: ใช้ Key Pair เดิมจาก Lab 2

```
ls -l ~/.ssh/lab2-key.pem
chmod 400 ~/.ssh/lab2-key.pem
```

แบบ B: สร้าง Key Pair ใหม่ผ่าน AWS Console

1. เข้า AWS Console ด้วยบัญชีที่จะใช้จริง ตรวจมุมขวาบนว่าเป็น **Asia Pacific (Singapore)**
2. ไปที่ **EC2** → **Network & Security** → **Key Pairs** → **Create key pair**
3. กรอกค่า
   - Name: `coffee-<รหัสนักศึกษา>-key` เช่น `coffee-b6612345-key`
   - Key pair type: **RSA**
   - Private key file format: **.pem**
4. กด **Create key pair** เบราว์เซอร์จะดาวน์โหลดไฟล์ `coffee-b6612345-key.pem` ลง Downloads **ไฟล์นี้ดาวน์โหลดได้ครั้งเดียว** ถ้าหายต้องสร้างใหม่
5. คัดลอกเข้า Ubuntu WSL และตั้งสิทธิ์

   ```
   mkdir -p ~/.ssh
   cp "/mnt/c/Users/<ชื่อผู้ใช้ Windows>/Downloads/coffee-b6612345-key.pem" ~/.ssh/
   chmod 400 ~/.ssh/coffee-b6612345-key.pem
   ls -l ~/.ssh/coffee-b6612345-key.pem     # ต้องขึ้นต้นด้วย -r--------
   ```

6. แก้ `ansible/vars.yml` ให้ตรงกับ key ใหม่

   ```
   key_name: "coffee-b6612345-key"
   ssh_key_file: "{{ lookup('ansible.builtin.env', 'HOME') }}/.ssh/coffee-b6612345-key.pem"
   ```

แบบ B ทางลัด: สร้างด้วยคำสั่งเดียวจาก WSL (หลังลงชื่อเข้า AWS แล้ว ได้ผลเหมือนข้อ 1–5)

```
aws ec2 create-key-pair --key-name coffee-b6612345-key --key-type rsa --key-format pem \
  --query KeyMaterial --output text > ~/.ssh/coffee-b6612345-key.pem
chmod 400 ~/.ssh/coffee-b6612345-key.pem
```

ข้อควรระวัง

- ห้ามส่งไฟล์ `.pem` ในแชท หรือใส่ใน Git (`.gitignore` กัน `*.pem` ไว้แล้ว) ใครมีไฟล์นี้เข้าเครื่องได้ทันที
- ถ้าเปลี่ยนบัญชี AWS ค่าอื่นใน `vars.yml` ต้องเปลี่ยนตามด้วย ได้แก่ `owner_id`, `vpc_id`, `subnet_id` (ส่น `ami_id` ใช้ค่าเดิมได้ถ้าอยู่ Region เดียวกัน) และต้อง `aws login` ด้วยบัญชีใหม่

8.5 กรอกค่าใน vars.yml และ secrets.yml

เปิด `ansible/vars.yml` แล้วแก้ทุกค่าที่เขียนว่า `REPLACE` หรือ `bXXXXXXXX`

ค่า	ตัวอย่าง	หาได้ที่ไหน
`owner_id`	`b6612345`	รหัสนักศึกษาของเจ้าของบัญชี AWS

ค่า	ตัวอย่าง	หาได้ที่ไหน
`vpc_id`	`vpc-0abc...`	VPC → Your VPCs → VPC ID (ใช้ Default VPC ได้)
`subnet_id`	`subnet-0abc...`	VPC → Subnets → Subnet ใน VPC เดียวกันที่ออกอินเทอร์เน็ตได้
`ami_id`	`ami-0abc...`	EC2 → Launch instance → Ubuntu Server 24.04 LTS, 64-bit (x86)
`key_name`	`coffee-b6612345-key`	ชื่อ Key Pair จากข้อ 8.4 (ไม่มี .pem)
`ssh_key_file`	`~/.ssh/coffee-b6612345-key.pem`	ไฟล์ .pem ใน WSL จากข้อ 8.4
`server_type`, `worker_type`	`t3.small`	ตรวจราคาและ Free Tier ของบัญชีก่อน
`worker_count`	`2`	จำนวน worker (1 หรือ 2)
`dockerhub_user`	`boss123`	username Docker Hub (ข้อ 8.6)
`app_version`	`1.0.0`	เวอร์ชัน image ที่ push ไว้

`ssh_cidr` ไม่ต้องแก้ มันหา IP ของเครือข่ายที่ใช้อยู่ให้เอง ถ้าย้ายไปใช้ Wi-Fi อื่นให้รัน `provision.yml` ใหม่เพื่ออัปเดตกฎ SSH

```
cp secrets.example.yml secrets.yml
nano secrets.yml                          # ตั้ง db_password และ staff_pin เอง
ansible-playbook --syntax-check provision.yml   # ต้องขึ้น playbook: provision.yml
```

8.6 build image แล้ว push ขึ้น Docker Hub

EC2 จะดึง image จาก Docker Hub ไปรัน จึงต้องส่ง image ของ frontend และ backend ขึ้นไปก่อน (Database ใช้ `postgres:16` สำเร็จรูป)

```
docker login
cd ~/coffee-queue
./scripts/build-push.sh boss123 1.0.0        # แทน boss123 ด้วย username ของเรา
```

ถ้าใช้ PowerShell บน Windows ให้รัน 2 บรรทัดนี้ในโฟลเดอร์ `coffee-queue/app` แทน

```
docker buildx build --platform linux/amd64 -t boss123/coffee-frontend:1.0.0 --push ./frontend
docker buildx build --platform linux/amd64 -t boss123/coffee-backend:1.0.0 --push ./backend
```

เปิด hub.docker.com ต้องเห็น `coffee-frontend` และ `coffee-backend` แท็ก `1.0.0` และตั้งเป็น **Public** `--platform linux/amd64` สำคัญมากสำหรับคนใช้ Mac M1/M2/M3 เพราะ EC2 เป็น x86

8.7 ลำดับการรัน

ทุกคำสั่งรันใน `~/coffee-queue/ansible` หลังตั้งค่าข้อ 8.3 แล้ว ดูบรรทัด `PLAY RECAP` ต้องเป็น `failed=0`

ขั้น	คำสั่ง	ผลที่ควรได้
1. สร้าง EC2	`ansible-playbook provision.yml`	EC2 3 เครื่อง Running, มี `inventory.ini`, แสดง Instance ID
2. ทดสอบเชื่อมต่อ	`ansible -i inventory.ini all -m ansible.builtin.ping`	`pong` ครบ 3 เครื่อง
3. ติดตั้ง k3s	`ansible-playbook -i inventory.ini k3s.yml`	node 3 ตัวสถานะ `Ready`
4. deploy แอป	`ansible-playbook -i inventory.ini deploy.yml`	pod ทุกตัว `Running`, บอกลิงก์ `http://<Public IP>/`

เข้าไปดู cluster เอง:

```
ssh -i ~/.ssh/coffee-b6612345-key.pem ubuntu@<Public IP ของ server1>
kubectl get nodes
kubectl -n coffee-queue get pods -o wide
kubectl -n coffee-queue get svc,ingress,hpa
```

8.8 ทดสอบบน AWS

`kubectl` รันบน server1 ส่วนคำสั่งอื่นรันบน WSL ในโฟลเดอร์ `~/coffee-queue` แทน `<IP>` ด้วย Public IP เครื่องใดก็ได้

ข้อ	พิสูจน์	คำสั่ง	ผลที่ควรได้
1	Gateway	`curl http://<IP>/api/menu` และเปิด `http://<IP>/`	`/` ได้หน้าเว็บ, `/api` ได้ JSON
2	Load Balance	`./scripts/check-lb.sh <IP>`	`X-Served-By` สลับกันอย่างน้อย 2 ชื่อ pod
3	Scaling	บน server1: `kubectl -n coffee-queue get hpa backend -w` แล้วบน WSL: `docker run --rm -i -e TARGET=http://<IP> grafana/k6 run - < load-test/order-load.js`	REPLICAS เพิ่มจาก 2 สูงสุด 5 และลดกลับหลังจบประมาณ 5 นาที
4	Self-healing	`kubectl -n coffee-queue delete pod -l app=backend`	pod ใหม่ขึ้นแทนภายในไม่กี่วินาที
5	Persistence	สั่งเครื่องดื่ม แล้ว `kubectl -n coffee-queue delete pod postgres-0`	เลขคิวเดิมยังอยู่หลัง postgres-0 กลับมา

ข้อ	พิสูจน์	คำสั่ง	ผลที่ควรได้
6	Readiness	`kubectl -n coffee-queue scale statefulset postgres --replicas=0` แล้วกลับเป็น 1	backend READY 0/1 แต่ RESTARTS ไม่เพิ่ม
7	Rolling update	push `1.0.1`, แก้ `app_version`, รัน `deploy.yml` ซ้ำ; ย้อนด้วย `kubectl -n coffee-queue rollout undo deployment/frontend`	อัปเดตโดยเว็บไม่ล่ม ย้อนกลับได้
8	รันซ้ำ	`ansible-playbook provision.yml` อีกครั้ง	Instance ID เดิม ไม่สร้างเครื่องเกิน

บันทึกผลจริงและภาพหน้าจอใน `docs/test-results.md`

8.9 ลบทรัพยากรทั้งหมด

ทำทุกครั้งที่ทดสอบเสร็จ เพราะ EC2 คิดเงินตามเวลาที่เปิด

```
ansible-playbook destroy.yml
```

ตรวจใน AWS Console (Singapore): EC2 3 เครื่องเป็น Terminated, ไม่มี Security Group `coffee-<owner_id>-sg`, ไม่มี Volume ค้าง

8.10 Security Group ที่ provision.yml สร้าง

กฎ Inbound	Source	เหตุผล
SSH TCP 22	IP ผู้เรียน /32	ให้ Ansible เข้าไปติดตั้ง
HTTP TCP 80	0.0.0.0/0	ผู้ใช้เปิดเว็บผ่าน Ingress
ทุกพอร์ต (6443, 8472, 10250 ฯลฯ)	Security Group เดียวกัน	เครื่องใน cluster คุยกันได้

Database ไม่เปิดพอร์ตออกนอกเลย เข้าได้จาก Backend ผ่าน Service ภายใน cluster เท่านั้น

8.11 ปัญหาที่พบบ่อย (บน AWS)

อาการ	สาเหตุ	วิธีแก้
`Unable to locate credentials` / `NoRegion`	ไม่ได้ตั้ง AWS_PROFILE หรือ AWS_DEFAULT_REGION	ทำคำสั่ง “ทุกครั้งที่เปิด Terminal ใหม่” ในข้อ 8.3
`The refresh token has expired`	เซสชัน AWS หมดอายุ	`aws login --profile lab2 --region ap-southeast-1 --remote`

อาการ	สาเหตุ	วิธีแก้
`InvalidKeyPair.NotFound`	`key_name` ไม่มีในบัญชีหรือ Region นี้	สร้าง Key Pair ใหม่ใน Singapore ตามข้อ 8.4
`Permission denied (publickey)`	ไฟล์ .pem ไม่ตรงกับ `key_name` หรือสิทธิ์ไม่ใช่ 400	ตรวจ `ssh_key_file` และ `chmod 400`
assert ล้มใน `provision.yml`	ยังมี `REPLACE` ใน `vars.yml`	กรอกค่าให้ครบตามข้อ 8.5
node ไม่ครบ 3 ตัว	เครื่องเล็กเกินหรือเครื่องคุยกันไม่ได้	ใช้ `t3.small` ขึ้นไป แล้วรัน `k3s.yml` ซ้ำ
pod ค้าง `ImagePullBackOff`	ชื่อ image ผิด หรือ repo เป็น Private	ตรวจ `dockerhub_user`, `app_version` และตั้ง repo เป็น Public
pod ขึ้น `exec format error`	build บน Mac M1 โดยไม่ใส่ `--platform linux/amd64`	build ใหม่ด้วย `scripts/build-push.sh`
HPA ขึ้น `<unknown>/60%`	metrics ยังไม่มา	รอ 1–2 นาที ตรวจ `kubectl top pods -n coffee-queue`
HPA ไม่เพิ่ม pod	โหลดน้อยเกิน	เพิ่ม `target: 50` เป็น 100 ใน `order-load.js` หรือลด `averageUtilization` เป็น 30

ข้อควรระวังเรื่องค่าใช้จ่าย: EC2 3 เครื่องและ Public IPv4 คิดเงินตามเวลา ตั้ง AWS Budget ก่อนเริ่ม และรัน `destroy.yml` ทุกครั้ง โค้ดผ่าน syntax check และการตรวจ schema แล้ว แต่ยังไม่ได้รันบน AWS จริง

9. ภาคผนวก: โค้ดทุกไฟล์

ส่วนนี้คือโค้ดเต็มของทุกไฟล์ใน zip เรียงตามโฟลเดอร์ แต่ละไฟล์มีคำอธิบายสั้นๆ ว่าทำอะไร และคอมเมนต์ในโค้ดอธิบายบรรทัดสำคัญไว้แล้ว ไฟล์ที่**ต้องแก้**มีแค่ `ansible/vars.yml` และ `ansible/secrets.yml`

9.1 Frontend (app/frontend)

`app/frontend/Dockerfile`

สร้าง image ของหน้าเว็บ: เอา Nginx มาเป็นฐาน คัดลอกไฟล์ตั้งค่าและหน้าเว็บเข้าไป

```
FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY html/ /usr/share/nginx/html/
EXPOSE 80
```

`app/frontend/nginx.conf`

Nginx ส่งไฟล์หน้าเว็บ และส่ง `/api/` ต่อไป Backend (อธิบายในส่วนที่ 4.1)

```
server {
    listen 80;

    # หน้าเว็บ (HTML/CSS/JS)
    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    # ทุก request ที่ขึ้นต้นด้วย /api ส่งต่อไป Backend container
    location /api/ {
        proxy_pass http://backend:3000;
    }
}
```

`app/frontend/html/app.js`

ไฟล์กลางที่ทุกหน้าใช้: ตั้งค่า path ของ API, รายการสถานะ 4 ขั้น, เมนูสำรอง, ฟังก์ชันเรียก API (ใส่ PIN ใน header ให้) และสร้าง DOM แบบปลอดภัย

```
// ===== Coffee Queue: ตั้งค่าส่วนกลางของหน้าเว็บ =====
// ทุกหน้าเรียก Backend ผ่าน /api (Nginx ส่งต่อให้ container backend)
// ถ้า endpoint หรือชื่อฟิลด์ของ Backend เปลี่ยน แก้ที่ไฟล์นี้ไฟล์เดียวได้เลย
const API = '/api';

const ENDPOINTS = {
  menu:        () => '/menu',                                  // GET  -> [{id,name,price}]
  createOrder: () => '/orders',                                // POST {name, items:
[{drink,temp,quantity}]} -> {queue,items,total,...}
  getOrder:    (q) => '/orders/' + encodeURIComponent(q),      // GET  ->
{queue,status,name,items,total}
  staffOrders: () => '/staff/orders',                          // GET  (header X-Staff-PIN) ->
[order,...]
  setStatus:   (q) => '/orders/' + encodeURIComponent(q) + '/status', // PATCH {status} (header
```

```
X-Staff-PIN)
};

// key = ค่าสถานะที่ Backend เก็บ, label = ชื่อสถานะที่แสดง, desc = คำอธิบายสั้นใต้ชื่อ
// act = ข้อความบนปุ่มของพนักงานตอนเลื่อนออร์เดอร์ "ไปยัง" สถานะนั้น
const STATUSES = [
  { key: 'waiting', label: 'In queue',         desc: 'Order received',        icon: '🕒', act:
'In queue' },
  { key: 'making',  label: 'Being made',       desc: 'Your barista is on it', icon: '☕', act:
'Start making' },
  { key: 'ready',   label: 'Ready for pickup', desc: 'Come to the counter',   icon: '✅', act:
'Mark as ready' },
  { key: 'done',    label: 'Picked up',        desc: 'Enjoy your drink',      icon: '🛍️', act:
'Customer picked up' },
];

// เมนูสำรองเมื่อเรียก /api/menu ไม่ได้ (ราคาเป็นบาทต่อแก้ว)
const DEFAULT_MENU = [
  { name: 'Latte', price: 60 },
  { name: 'Americano', price: 50 },
  { name: 'Cappuccino', price: 60 },
  { name: 'Mocha', price: 65 },
  { name: 'Thai Tea', price: 45 },
  { name: 'Green Tea Latte', price: 55 },
  { name: 'Cocoa', price: 50 },
  { name: 'Chocolate', price: 55 },
];

// ===== ตัวช่วย =====
const norm = (s) => String(s || '').toLowerCase().replace(/[\s_-]+/g, '');
function statusIndex(s) {
  const n = norm(s);
  return STATUSES.findIndex((x) => norm(x.key) === n);
}
const queueOf = (o) => o.queue ?? o.queueNumber ?? o.queue_number ?? o.id;
const tempLabel = (t) => (norm(t) === 'hot' ? 'Hot' : 'Iced');
const baht = (n) => '฿' + Number(n).toLocaleString('en-US');

// ออร์เดอร์หนึ่งมีได้หลายรายการ: order.items = [{drink, temp, quantity}]
// รองรับรูปแบบเดิม (drink/temp/quantity ที่ระดับออร์เดอร์) ด้วย
function itemsOf(o) {
  if (Array.isArray(o.items) && o.items.length) return o.items;
  return o.drink ? [{ drink: o.drink, temp: o.temp, quantity: o.quantity ?? 1 }] : [];
}
function fillItems(ul, o) {
  ul.replaceChildren();
  itemsOf(o).forEach((it) => {
    ul.append(el('li', {},
      el('span', { class: 'nm' }, String(it.drink), it.temp ? el('span', { class: 'tp', text: ' '
+ tempLabel(it.temp) }) : ''),
      el('span', { class: 'qty', text: '× ' + (it.quantity ?? 1) })));
  });
}
// ยอดรวมของออร์เดอร์จาก Backend (ถ้ามี): total, totalPrice หรือ total_price
const priceOf = (o) => o.total ?? o.totalPrice ?? o.total_price ?? null;

async function api(path, { method = 'GET', body, pin } = {}) {
  const headers = {};
  if (body) headers['Content-Type'] = 'application/json';
  if (pin) headers['X-Staff-PIN'] = pin;
  let res;
```

```
  try {
    res = await fetch(API + path, { method, headers, body: body ? JSON.stringify(body) :
undefined });
  } catch (e) {
    throw Object.assign(new Error("Can't reach the server. Please try again."), { status: 0 });
  }
  if (!res.ok) {
    let msg = 'Something went wrong (' + res.status + ')';
    try {
      const j = await res.json();
      if (j && (j.message || j.error)) msg = j.message || j.error;
    } catch (e) { /* ignore */ }
    throw Object.assign(new Error(msg), { status: res.status });
  }
  if (res.status === 204) return null;
  try { return await res.json(); } catch (e) { return null; }
}

// สร้าง DOM แบบปลอดภัย (ใช้ textContent ไม่ใช้ innerHTML กับข้อมูลลูกค้า)
function el(tag, attrs, ...kids) {
  const n = document.createElement(tag);
  for (const [k, v] of Object.entries(attrs || {})) {
    if (k === 'class') n.className = v;
    else if (k === 'text') n.textContent = v;
    else if (k.startsWith('on')) n.addEventListener(k.slice(2), v);
    else n.setAttribute(k, v);
  }
  kids.flat().forEach((c) => n.append(c));
  return n;
}
function showMsg(box, text, type) {
  box.textContent = text || '';
  box.className = text ? 'msg ' + (type || 'err') : 'msg hidden';
}
```

app/frontend/html/index.html

หน้า Order drinks: โหลดเมนูจาก `GET /api/menu` เลือกได้หลายแก้วลงตะกร้า แล้วส่ง `POST /api/orders` ได้บัตรคิวกลับมา

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="theme-color" content="#123D2F">
  <title>Coffee Queue · Order</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?
family=IBM+Plex+Sans+Thai:wght@400;500;600&family=Mitr:wght@400;500&display=swap"
rel="stylesheet">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="top">
    <div class="bar">
      <span class="brand">Coffee Queue</span>
      <nav><a href="queue.html">Track order</a></nav>
```

```
    </div>
    <div class="intro">
      <h1>Order drinks</h1>
      <p>Add several drinks to one order and get a queue number.</p>
    </div>
  </header>

  <main>
    <form id="orderForm" autocomplete="off">
      <section class="card">
        <div class="group">
          <label class="field-title" for="name">Your name</label>
          <input type="text" id="name" maxlength="40" placeholder="e.g. Sam" required>
        </div>

        <div class="group">
          <span class="field-title">Drink</span>
          <div class="drinks" id="drinks"></div>
        </div>

        <div class="group">
          <span class="field-title">Hot or iced</span>
          <div class="pill" id="temps">
            <button type="button" class="opt on" data-v="hot">🔥 Hot</button>
            <button type="button" class="opt" data-v="iced">🧊 Iced</button>
          </div>
        </div>

        <div class="group">
          <span class="field-title">Quantity</span>
          <div class="stepper">
            <button type="button" id="minus" aria-label="Decrease quantity">−</button>
            <output id="qty">1</output>
            <button type="button" id="plus" aria-label="Increase quantity">+</button>
          </div>
        </div>

        <button class="btn ghost" id="addBtn" type="button" disabled>Add to order</button>
      </section>

      <section class="card" id="cartCard">
        <span class="field-title">Your order</span>
        <ul class="cart" id="cart"></ul>
        <p class="empty-cart" id="cartEmpty">Nothing here yet. Pick a drink above, then tap Add
to order.</p>
        <div class="total hidden" id="totalRow"><span>Total</span><span class="amt"
id="total">฿0</span></div>
        <div id="msg" class="msg hidden" style="margin:14px 0 0"></div>
      </section>

      <button class="btn cherry stick" id="submitBtn" type="submit" disabled>Place order</button>
    </form>

    <section id="result" class="ticket hidden">
      <div class="ticket-top">
        <p class="small">Your queue number</p>
        <p class="num" id="queueNum">-</p>
      </div>
      <div class="perf"></div>
      <div class="ticket-bottom">
        <ul class="lines" id="ticketLines"></ul>
```

```
        <p class="sub" id="nameLine"></p>
        <div class="total hidden" id="totalLine"><span>Total</span><span class="amt"
id="ticketTotal"></span></div>
        <a class="btn" id="trackLink" href="queue.html">Track my order</a>
        <button class="btn ghost" id="again" type="button">Order more</button>
      </div>
    </section>
  </main>

  <script src="app.js"></script>
  <script>
    // drink/price/temp/qty = สิ่งที่กำลังเลือกอยู่, items = รายการในออร์เดอร์
    const state = { drink: '', price: null, temp: 'hot', qty: 1, items: [] };
    const $ = (id) => document.getElementById(id);
    const MAX_QTY = 20;

    const cartTotal = () =>
      state.items.length && state.items.every((i) => i.price !== null)
        ? state.items.reduce((s, i) => s + i.price * i.quantity, 0)
        : null;

    function renderCart() {
      const ul = $('cart');
      ul.replaceChildren();
      state.items.forEach((it, idx) => {
        const minus = el('button', { type: 'button', 'aria-label': 'Decrease ' + it.drink, text:
'−' });
        const plus = el('button', { type: 'button', 'aria-label': 'Increase ' + it.drink, text:
'+' });
        const rm = el('button', { type: 'button', class: 'rm', text: 'Remove' });
        minus.addEventListener('click', () => {
          if (it.quantity > 1) it.quantity -= 1; else state.items.splice(idx, 1);
          refresh();
        });
        plus.addEventListener('click', () => { it.quantity = Math.min(MAX_QTY, it.quantity + 1);
refresh(); });
        rm.addEventListener('click', () => { state.items.splice(idx, 1); refresh(); });
        ul.append(el('li', {},
          el('span', { class: 'nm' }, it.drink, el('span', { class: 'tp', text: ' ' +
tempLabel(it.temp) })),
          el('span', { class: 'lt', text: it.price !== null ? baht(it.price * it.quantity) : ''
}),
          el('span', { class: 'mini' }, minus, el('output', { text: String(it.quantity) }),
plus),
          rm));
      });
    }

    function refresh() {
      renderCart();
      $('cartEmpty').classList.toggle('hidden', state.items.length > 0);
      const total = cartTotal();
      $('totalRow').classList.toggle('hidden', total === null);
      if (total !== null) $('total').textContent = baht(total);

      const add = $('addBtn');
      add.disabled = !state.drink;
      add.textContent = 'Add to order' + (state.drink && state.price !== null ? ' ' +
baht(state.price * state.qty) : '');
      $('submitBtn').disabled = !(state.items.length || state.drink);
    }
```

```
    function addCurrent() {
      if (!state.drink) return false;
      const same = state.items.find((i) => i.drink === state.drink && i.temp === state.temp);
      if (same) same.quantity = Math.min(MAX_QTY, same.quantity + state.qty);
      else state.items.push({ drink: state.drink, price: state.price, temp: state.temp, quantity:
state.qty });
      state.drink = ''; state.price = null; state.qty = 1;
      $('qty').textContent = '1';
      $('drinks').querySelectorAll('.drink').forEach((c) => { c.classList.remove('on');
c.setAttribute('aria-pressed', 'false'); });
      return true;
    }

    function renderDrinks(list) {
      const box = $('drinks');
      box.replaceChildren();
      list.forEach((d) => {
        const name = typeof d === 'string' ? d : (d.name || d.title);
        if (!name) return;
        const price = typeof d === 'object' && d.price != null && !isNaN(Number(d.price)) ?
Number(d.price) : null;
        const btn = el('button', { type: 'button', class: 'drink', 'aria-pressed': 'false' },
          el('span', { class: 'nm' },
            el('span', { text: name }),
            price !== null ? el('span', { class: 'pr', text: baht(price) }) : ''),
          el('span', { class: 'tick' }));
        btn.addEventListener('click', () => {
          const was = state.drink === name; // กดซ้ำเพื่อยกเลิกการเลือก
          box.querySelectorAll('.drink').forEach((c) => { c.classList.remove('on');
c.setAttribute('aria-pressed', 'false'); });
          state.drink = was ? '' : name;
          state.price = was ? null : price;
          if (!was) { btn.classList.add('on'); btn.setAttribute('aria-pressed', 'true'); }
          refresh();
        });
        box.append(btn);
      });
    }

    async function loadMenu() {
      let list = DEFAULT_MENU;
      try {
        const data = await api(ENDPOINTS.menu());
        const arr = Array.isArray(data) ? data : (data && data.items);
        if (Array.isArray(arr) && arr.length) list = arr;
      } catch (e) { /* ใช้เมนูเริ่มต้น */ }
      renderDrinks(list);
    }

    $('temps').addEventListener('click', (e) => {
      const b = e.target.closest('.opt');
      if (!b) return;
      state.temp = b.dataset.v;
      $('temps').querySelectorAll('.opt').forEach((c) => c.classList.toggle('on', c === b));
    });
    $('minus').onclick = () => { state.qty = Math.max(1, state.qty - 1); $('qty').textContent =
state.qty; refresh(); };
    $('plus').onclick = () => { state.qty = Math.min(MAX_QTY, state.qty + 1);
$('qty').textContent = state.qty; refresh(); };
```

```
    $('addBtn').addEventListener('click', () => {
      if (!addCurrent()) return;
      showMsg($('msg'), '');
      refresh();
      const reduce = window.matchMedia('(prefers-reduced-motion: reduce)').matches;
      $('cartCard').scrollIntoView({ behavior: reduce ? 'auto' : 'smooth', block: 'nearest' });
    });

    $('orderForm').addEventListener('submit', async (e) => {
      e.preventDefault();
      addCurrent(); // เครื่องดื่มที่เลือกค้างไว้ จะรวมในออร์เดอร์ด้วย
      refresh();
      const name = $('name').value.trim();
      if (!name) return showMsg($('msg'), 'Enter your name so the barista can call you.');
      if (!state.items.length) return showMsg($('msg'), 'Pick at least one drink.');
      showMsg($('msg'), '');
      const btn = $('submitBtn');
      btn.disabled = true; btn.textContent = 'Sending order...';
      const items = state.items.map((i) => ({ drink: i.drink, temp: i.temp, quantity: i.quantity
}));
      try {
        const res = await api(ENDPOINTS.createOrder(), { method: 'POST', body: { name, items }
});
        const q = res && queueOf(res);
        if (q === undefined || q === null) throw new Error('No queue number came back. Please try
again.');
        try { localStorage.setItem('lastQueue', String(q)); } catch (_) {}
        $('queueNum').textContent = q;
        fillItems($('ticketLines'), { items: (res.items && res.items.length) ? res.items : items
});
        $('nameLine').textContent = 'For ' + name;
        // ใช้ยอดที่ Backend คำนวณถ้ามี ไม่งั้นรวมจากราคาในเมนู
        const total = priceOf(res) ?? cartTotal();
        $('totalLine').classList.toggle('hidden', total === null);
        if (total !== null) $('ticketTotal').textContent = baht(total);
        $('trackLink').href = 'queue.html?q=' + encodeURIComponent(q);
        state.items = [];
        $('orderForm').classList.add('hidden');
        const t = $('result');
        t.classList.remove('hidden', 'print');
        void t.offsetWidth; // เริ่มอนิเมชันใหม่ทุกครั้งที่สั่ง
        t.classList.add('print');
        window.scrollTo({ top: 0 });
      } catch (err) {
        showMsg($('msg'), err.message);
      } finally {
        btn.textContent = 'Place order';
        refresh();
      }
    });

    $('again').onclick = () => {
      $('result').classList.add('hidden');
      $('orderForm').classList.remove('hidden');
      refresh();
      window.scrollTo({ top: 0 });
    };

    loadMenu();
    refresh();
  </script>
```

```
</body>
</html>
```

app/frontend/html/queue.html

หน้า Track order: ใส่เลขคิวแล้วเรียก `GET /api/orders/A015` แสดงบัตรคิวและ timeline สถานะ อัปเดตเองทุก 8 วินาที

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="theme-color" content="#123D2F">
  <title>Coffee Queue · Track order</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?
family=IBM+Plex+Sans+Thai:wght@400;500;600&family=Mitr:wght@400;500&display=swap"
rel="stylesheet">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="top">
    <div class="bar">
      <span class="brand">Coffee Queue</span>
      <nav><a href="index.html">Order drinks</a></nav>
    </div>
    <div class="intro">
      <h1>Track your order</h1>
      <p>Enter your queue number to see how your drinks are doing.</p>
    </div>
  </header>

  <main>
    <form id="form" class="card" autocomplete="off">
      <label class="field-title" for="q">Queue number</label>
      <div class="row">
        <input type="text" id="q" placeholder="e.g. A012" required>
        <button class="btn cherry" type="submit">Check</button>
      </div>
      <div id="msg" class="msg hidden" style="margin:14px 0 0"></div>
    </form>

    <section id="result" class="ticket hidden" data-s="0">
      <div class="ticket-top">
        <p class="small">Queue</p>
        <p class="num" id="qNum"></p>
        <p class="state" id="stateText"></p>
      </div>
      <div class="perf"></div>
      <div class="ticket-bottom">
        <ul class="lines" id="lines"></ul>
        <p class="sub" id="nameLine"></p>
        <div class="total hidden" id="totalLine"><span>Total</span><span class="amt"
id="ticketTotal"></span></div>
        <ul class="steps" id="steps"></ul>
        <p id="hint" class="hint"></p>
      </div>
```

```
    </section>
  </main>

  <script src="app.js"></script>
  <script>
    const $ = (id) => document.getElementById(id);
    let timer = null;
    let current = '';

    function render(order, q) {
      const idx = statusIndex(order.status);
      const t = $('result');
      t.dataset.s = idx;
      $('qNum').textContent = queueOf(order) ?? q;
      $('stateText').textContent = idx >= 0 ? `${STATUSES[idx].icon} ${STATUSES[idx].label}` :
String(order.status || 'Unknown status');
      fillItems($('lines'), order);
      $('nameLine').textContent = order.name ? 'For ' + order.name : '';
      const tot = priceOf(order);
      $('totalLine').classList.toggle('hidden', tot === null);
      if (tot !== null) $('ticketTotal').textContent = baht(tot);

      const ul = $('steps');
      ul.replaceChildren();
      STATUSES.forEach((s, i) => {
        const cls = i < idx ? 'done' : i === idx ? 'now' : '';
        ul.append(el('li', { class: cls },
          el('span', { class: 'ic', text: i < idx ? '✓' : s.icon }),
          el('span', {}, el('span', { class: 'th', text: s.label }), el('span', { class: 'en',
text: s.desc }))));
      });

      if (idx === 2) $('hint').textContent = 'Your drinks are ready. Please pick them up at the
counter.';
      else if (idx === 3) $('hint').textContent = 'Picked up. Thanks for stopping by!';
      else if (idx === -1) $('hint').textContent = '';
      else $('hint').textContent = 'This page updates every 8 seconds.';

      if (t.classList.contains('hidden')) {
        t.classList.remove('hidden');
        t.classList.add('print');
      }
      return idx;
    }

    async function check(q, silent) {
      clearTimeout(timer);
      current = q;
      if (!silent) showMsg($('msg'), '');
      try {
        const order = await api(ENDPOINTS.getOrder(q));
        if (!order) throw Object.assign(new Error('Queue number not found'), { status: 404 });
        const idx = render(order, q);
        if (q === current && idx !== 3) timer = setTimeout(() => check(q, true), 8000);
      } catch (err) {
        if (err.status === 404) {
          $('result').classList.add('hidden');
          $('result').classList.remove('print');
          showMsg($('msg'), 'Queue number not found. Please check the number.');
        } else if (!silent) {
          showMsg($('msg'), err.message);
```

```
        } else {
          timer = setTimeout(() => check(q, true), 8000);
        }
      }
    }

    $('form').addEventListener('submit', (e) => {
      e.preventDefault();
      const q = $('q').value.trim();
      if (q) check(q, false);
    });

    (function init() {
      const p = new URLSearchParams(location.search).get('q');
      let saved = '';
      try { saved = localStorage.getItem('lastQueue') || ''; } catch (_) {}
      const q = p || saved;
      if (q) { $('q').value = q; check(q, false); }
    })();
  </script>
</body>
</html>
```

`app/frontend/html/staff.html`

หน้า Staff: ใส่ PIN แล้วดึง `GET /api/staff/orders` แยกแท็บตามสถานะ กดปุ่มเพื่อ `PATCH /api/orders/A015/status` ไปขั้นถัดไป อัปเดตเองทุก 10 วินาที

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="theme-color" content="#123D2F">
  <meta name="robots" content="noindex">
  <title>Coffee Queue · Staff</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?
family=IBM+Plex+Sans+Thai:wght@400;500;600&family=Mitr:wght@400;500&display=swap"
rel="stylesheet">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header class="top">
    <div class="bar">
      <span class="brand">Coffee Queue</span>
      <nav><a href="#" id="logout" class="hidden">Log out</a></nav>
    </div>
    <div class="intro">
      <h1>Staff</h1>
      <p>See incoming orders and move them to the next step.</p>
    </div>
  </header>

  <main>
    <form id="login" class="card" autocomplete="off">
      <label class="field-title" for="pin">Staff PIN</label>
      <input type="password" id="pin" inputmode="numeric" placeholder="Enter PIN" required>
```

```
      <div id="loginMsg" class="msg hidden" style="margin:14px 0 0"></div>
      <button class="btn cherry" type="submit" style="margin-top:16px">Log in</button>
    </form>

    <section id="board" class="hidden">
      <div class="toolbar">
        <span class="updated" id="updated"></span>
        <button class="btn small ghost" id="refresh" type="button">Refresh</button>
      </div>
      <div class="tabs" id="tabs" role="tablist"></div>
      <div id="boardMsg" class="msg hidden"></div>
      <div id="list"></div>
      <div id="empty" class="empty hidden">
        <strong>No orders here</strong>
        New orders show up here within 10 seconds.
      </div>
    </section>
  </main>

  <script src="app.js"></script>
  <script>
    const $ = (id) => document.getElementById(id);
    const FILTERS = [
      { k: 'open', label: 'To do' },
      { k: 0, label: STATUSES[0].label },
      { k: 1, label: STATUSES[1].label },
      { k: 2, label: STATUSES[2].label },
      { k: 3, label: STATUSES[3].label },
    ];
    let pin = '';
    let timer = null;
    let orders = [];
    let filter = 'open';

    const matches = (o, k) => (k === 'open' ? statusIndex(o.status) !== 3 : statusIndex(o.status)
=== k);

    function logout(message) {
      clearTimeout(timer);
      pin = '';
      try { sessionStorage.removeItem('staffPin'); } catch (_) {}
      $('board').classList.add('hidden');
      $('login').classList.remove('hidden');
      $('logout').classList.add('hidden');
      $('pin').value = '';
      showMsg($('loginMsg'), message || '', 'err');
    }

    function renderTabs() {
      const box = $('tabs');
      box.replaceChildren();
      FILTERS.forEach((f) => {
        const n = orders.filter((o) => matches(o, f.k)).length;
        const b = el('button', { type: 'button', class: 'tab' + (f.k === filter ? ' on' : ''),
role: 'tab', 'aria-selected': String(f.k === filter) },
          f.label, el('span', { class: 'n', text: String(n) }));
        b.addEventListener('click', () => { filter = f.k; render(); });
        box.append(b);
      });
    }
```

```
    function render() {
      renderTabs();
      const list = $('list');
      list.replaceChildren();
      const rows = orders.filter((o) => matches(o, filter));
      $('empty').classList.toggle('hidden', rows.length > 0);

      rows.forEach((o) => {
        const q = queueOf(o);
        const idx = statusIndex(o.status);
        const next = idx >= 0 && idx < STATUSES.length - 1 ? STATUSES[idx + 1] : null;
        const lines = el('ul', { class: 'lines' });
        fillItems(lines, o);
        const card = el('div', { class: 'card order s' + Math.max(idx, 0) },
          el('div', { class: 'head' },
            el('span', { class: 'q', text: String(q) }),
            el('span', { class: 'badge', text: idx >= 0 ? `${STATUSES[idx].icon}
${STATUSES[idx].label}` : String(o.status || '?') })),
          lines,
          el('div', { class: 'meta' },
            el('span', { class: 'who', text: o.name ? 'For ' + o.name : '' }),
            priceOf(o) !== null ? el('span', { class: 'amt', text: baht(priceOf(o)) }) : ''));
        if (next) {
          const btn = el('button', { class: 'btn act' + (idx + 1), type: 'button', text: next.act
});
          btn.addEventListener('click', () => advance(q, next.key, btn));
          card.append(btn);
        }
        list.append(card);
      });
    }

    async function load(silent) {
      clearTimeout(timer);
      try {
        const data = await api(ENDPOINTS.staffOrders(), { pin });
        orders = Array.isArray(data) ? data : (data && data.orders) || [];
        showMsg($('boardMsg'), '');
        $('updated').textContent = 'Updated ' + new Date().toLocaleTimeString('en-GB', { hour:
'2-digit', minute: '2-digit' });
        render();
      } catch (err) {
        if (err.status === 401 || err.status === 403) return logout('Wrong PIN. Please try
again.');
        showMsg($('boardMsg'), err.message);
        if (!silent && !orders.length) throw err;
      }
      timer = setTimeout(() => load(true), 10000);
    }

    async function advance(q, status, btn) {
      btn.disabled = true;
      try {
        await api(ENDPOINTS.setStatus(q), { method: 'PATCH', body: { status }, pin });
        const o = orders.find((x) => String(queueOf(x)) === String(q));
        if (o) o.status = status;
        render();
        load(true);
      } catch (err) {
        if (err.status === 401 || err.status === 403) return logout('Wrong PIN. Please try
again.');
```

```
        btn.disabled = false;
        showMsg($('boardMsg'), err.message);
      }
    }

    async function enter(p) {
      pin = p;
      try {
        await load(false);
        if (!pin) return; // ถูกออกจากระบบเพราะ PIN ผิด
        try { sessionStorage.setItem('staffPin', p); } catch (_) {}
        $('login').classList.add('hidden');
        $('board').classList.remove('hidden');
        $('logout').classList.remove('hidden');
      } catch (err) {
        logout(err.message);
      }
    }

    $('login').addEventListener('submit', (e) => {
      e.preventDefault();
      showMsg($('loginMsg'), '');
      const p = $('pin').value.trim();
      if (p) enter(p);
    });
    $('logout').addEventListener('click', (e) => { e.preventDefault(); logout(''); });
    $('refresh').addEventListener('click', () => load(true));

    (function init() {
      let saved = '';
      try { saved = sessionStorage.getItem('staffPin') || ''; } catch (_) {}
      if (saved) enter(saved);
    })();
  </script>
</body>
</html>
```

app/frontend/html/style.css

หน้าตาของทั้ง 3 หน้า ธีมเขียว-แดงเชอร์รี่ ข้อความภาษาอังกฤษ ออกแบบให้ใช้บนมือถือ (ช่องกรอก 16px กัน iPhone ซูมเอง)

```
:root{
  --leaf:#123D2F; --leaf-2:#1E5A45;
  --cherry:#D7263D; --cherry-d:#B81E32;
  --page:#EAF1EC; --card:#fff; --ink:#14251E; --muted:#5F7369; --line:#D6E2DA; --field:#F2F6F3;
  --cold:#2C7BA0; --ready:#17784C; --wait:#B3690A; --done:#5F7369;
  --r-lg:22px; --r-md:14px;
  --display:"Mitr","Prompt","Noto Sans Thai",system-ui,sans-serif;
  --body:"IBM Plex Sans Thai","Noto Sans Thai",system-ui,-apple-system,"Segoe UI",sans-serif;
}
*{box-sizing:border-box}
html{-webkit-text-size-adjust:100%}
body{margin:0;background:var(--page);color:var(--ink);font-family:var(--body);font-
size:17px;line-height:1.55}
button,input{font-family:inherit}
:focus-visible{outline:3px solid var(--cherry);outline-offset:2px}
.hidden{display:none!important}
.muted{color:var(--muted)}
```

```
/* ---------- หัวหน้า ---------- */
header.top{background:var(--leaf);color:#fff;padding:14px 18px 62px;border-radius:0 0 30px 30px}
header.top .bar{max-width:520px;margin:0 auto;display:flex;align-items:center;justify-
content:space-between;gap:10px}
.brand{font-family:var(--display);font-size:1.05rem;display:flex;align-items:center;gap:9px}
.brand::before{content:"";width:15px;height:15px;border-radius:50%;background:var(--cherry);box-
shadow:inset -3px -3px 0 rgba(0,0,0,.18)}
header.top nav a{color:#fff;text-decoration:none;font-size:.9rem;padding:6px 15px;border-
radius:999px;box-shadow:inset 0 0 0 1.5px rgba(255,255,255,.4)}
header.top .intro{max-width:520px;margin:26px auto 0}
header.top h1{font-family:var(--display);font-weight:500;font-size:2.1rem;line-
height:1.15;margin:0 0 6px}
header.top p{margin:0;color:rgba(255,255,255,.78);font-size:.98rem}
main{max-width:520px;margin:-36px auto 0;padding:0 16px 44px;position:relative}

/* ---------- การ์ดและฟอร์ม ---------- */
.card{background:var(--card);border-radius:var(--r-lg);padding:20px 18px;margin-bottom:16px;box-
shadow:0 14px 28px -18px rgba(18,61,47,.45)}
.group{margin-bottom:22px}
.field-title{display:block;font-family:var(--display);font-weight:500;font-
size:1.02rem;color:var(--leaf);margin:0 0 10px}
input[type=text],input[type=password]{width:100%;font-size:16px;padding:14px 16px;border:2px
solid transparent;border-radius:var(--r-md);background:var(--field);color:var(--ink)}
input::placeholder{color:#8DA198}
input:focus{outline:none;border-color:var(--cherry);background:#fff}

.drinks{display:grid;grid-template-columns:1fr 1fr;gap:10px}
.drink{display:flex;align-items:center;justify-content:space-between;gap:8px;min-
height:60px;padding:12px 14px;text-align:left;font-size:1rem;font-weight:500;color:var(--
ink);background:var(--field);border:2px solid transparent;border-radius:var(--r-
md);cursor:pointer;transition:background .15s,color .15s}
.drink .tick{flex:none;width:22px;height:22px;border-radius:50%;border:2px solid var(--
line);background:#fff;display:flex;align-items:center;justify-content:center;font-
size:13px;color:#fff}
.drink.on{background:var(--leaf);color:#fff}
.drink.on .tick{background:var(--cherry);border-color:var(--cherry)}
.drink.on .tick::after{content:"✓"}

.drink .nm{display:flex;flex-direction:column;gap:2px;line-height:1.25;min-width:0}
.drink .pr{font-size:.85rem;font-weight:400;color:var(--muted)}
.drink.on .pr{color:rgba(255,255,255,.8)}
.total{display:flex;align-items:baseline;justify-content:space-between;gap:10px;margin:0 0
14px;padding-top:14px;border-top:2px dashed var(--line)}
.total > span:first-child{color:var(--muted)}
.amt{font-family:var(--display);font-size:1.6rem;line-height:1.2;color:var(--cherry);font-
variant-numeric:tabular-nums}
.ticket-bottom .total{margin:14px 0 0;padding-top:12px}
.lines{list-style:none;margin:0;padding:0;display:grid;gap:6px}
.lines li{display:flex;align-items:baseline;justify-content:space-between;gap:12px}
.lines .nm{font-family:var(--display);font-size:1.12rem;line-height:1.35}
.lines .tp{font-family:var(--body);font-size:.92rem;color:var(--muted)}
.lines .qty{color:var(--muted);white-space:nowrap}
.ticket-bottom .lines{margin-bottom:6px}
.order .lines{margin:8px 0 6px}
.order .lines .nm{font-family:var(--body);font-weight:600;font-size:1.02rem}

.cart{list-style:none;margin:0 0 4px;padding:0}
.cart li{display:grid;grid-template-columns:1fr auto;gap:8px 12px;align-items:center;padding:12px
0;border-bottom:1px solid var(--line)}
.cart li:last-child{border-bottom:0}
```

```
.cart .nm{font-weight:600}
.cart .tp{color:var(--muted);font-weight:400}
.cart .lt{font-family:var(--display);color:var(--leaf)}
.cart .mini{display:inline-flex;align-items:center;justify-self:start;background:var(--
field);border-radius:999px;padding:3px}
.cart .mini button{width:36px;height:36px;border-radius:50%;border:0;background:#fff;color:var(--
leaf);font-size:1.2rem;line-height:1;cursor:pointer;box-shadow:0 1px 4px -1px rgba(18,61,47,.35)}
.cart .mini output{min-width:38px;text-align:center;font-family:var(--display)}
.cart .rm{justify-self:end;border:0;background:transparent;color:var(--cherry-d);font-
size:.92rem;font-weight:500;padding:8px 4px;cursor:pointer}
.empty-cart{margin:0 0 14px;color:var(--muted);font-size:.95rem}
form#orderForm .stick{margin-top:2px}
.pill{display:grid;grid-template-columns:1fr 1fr;gap:4px;padding:4px;background:var(--
field);border-radius:999px}
.pill .opt{border:0;background:transparent;border-radius:999px;padding:12px;font-size:1rem;font-
weight:500;color:var(--muted);cursor:pointer;transition:background .15s,color .15s}
.pill .opt.on[data-v="hot"]{background:var(--cherry);color:#fff}
.pill .opt.on[data-v="iced"]{background:var(--cold);color:#fff}

.stepper{display:inline-flex;align-items:center;background:var(--field);border-
radius:999px;padding:4px}
.stepper button{width:48px;height:48px;border-radius:50%;border:0;background:#fff;color:var(--
leaf);font-size:1.6rem;line-height:1;cursor:pointer;box-shadow:0 2px 6px -2px rgba(18,61,47,.35)}
.stepper button:active{background:var(--line)}
.stepper output{min-width:64px;text-align:center;font-family:var(--display);font-
size:1.5rem;color:var(--leaf)}

.btn{display:block;width:100%;border:0;border-radius:16px;padding:16px;background:var(--
leaf);color:#fff;font:500 1.08rem var(--display);text-align:center;text-
decoration:none;cursor:pointer;transition:transform .12s,background .12s}
.btn:active{transform:scale(.98)}
.btn.cherry{background:var(--cherry);box-shadow:0 12px 22px -12px rgba(215,38,61,.8)}
.btn.cherry:active{background:var(--cherry-d)}
.btn.ghost{background:transparent;color:var(--leaf);box-shadow:inset 0 0 0 2px var(--leaf)}
.btn.small{display:inline-block;width:auto;padding:9px 18px;font-size:.95rem;border-radius:999px}
.btn:disabled{opacity:.5;cursor:not-allowed}
.stick{position:sticky;bottom:12px}
.row{display:flex;gap:10px}
.row input{flex:2}
.row .btn{flex:1;width:auto}

.msg{padding:12px 14px;border-radius:var(--r-md);margin:0 0 14px;font-size:.95rem}
.msg.err{background:#FCE8EA;color:#9B1B2C}
.msg.info{background:#E3F3EA;color:var(--ready)}

/* ---------- บัตรคิว ---------- */
.ticket{position:relative;background:var(--card);border-radius:var(--r-
lg);overflow:hidden;margin-bottom:16px;box-shadow:0 18px 30px -20px rgba(18,61,47,.6)}
.ticket-top{background:var(--cherry);color:#fff;text-align:center;padding:24px 18px
30px;transition:background .3s}
.ticket[data-s="0"] .ticket-top{background:var(--wait)}
.ticket[data-s="2"] .ticket-top{background:var(--ready)}
.ticket[data-s="3"] .ticket-top{background:var(--done)}
.ticket-top .small{margin:0;opacity:.88;font-size:.95rem}
.ticket-top .num{font-family:var(--display);font-size:4.2rem;line-height:1.05;letter-
spacing:.02em;margin:2px 0 0;word-break:break-all}
.ticket-top .state{font-family:var(--display);font-size:1.15rem;margin:8px 0 0}
.perf{position:relative;height:0;border-top:2px dashed var(--line)}
.perf::before,.perf::after{content:"";position:absolute;top:-13px;width:24px;height:24px;border-
radius:50%;background:var(--page)}
.perf::before{left:-12px}
```

```
.perf::after{right:-12px}
.ticket-bottom{padding:22px 18px 20px}
.ticket-bottom .item{font-family:var(--display);font-size:1.2rem;line-height:1.35;margin:0}
.ticket-bottom .sub{margin:2px 0 0;color:var(--muted)}
.ticket-bottom .hint{margin:14px 0 0;color:var(--muted);font-size:.93rem}
.ticket-bottom .btn{margin-top:16px}
.ticket-bottom .btn + .btn{margin-top:10px}
.ticket.print{animation:print .85s cubic-bezier(.2,.8,.2,1) backwards}
@keyframes print{from{clip-path:inset(0 0 100% 0);transform:translateY(-14px)}to{clip-
path:inset(0 0 0 0);transform:none}}

/* ---------- เส้นเวลาสถานะ ---------- */
.steps{list-style:none;padding:0;margin:18px 0 0}
.steps li{position:relative;display:flex;align-items:center;gap:14px;padding:11px 8px;color:var(-
-muted)}
.steps li:not(:last-
child)::after{content:"";position:absolute;left:25.5px;top:47px;width:3px;height:22px;background:
var(--line);border-radius:2px}
.steps li.done:not(:last-child)::after{background:var(--ready)}
.steps .ic{flex:none;width:36px;height:36px;border-radius:50%;background:var(--
field);display:flex;align-items:center;justify-content:center;font-size:1.05rem}
.steps li.done{color:var(--ink)}
.steps li.done .ic{background:var(--ready);color:#fff;font-size:.95rem}
.steps li.now{color:var(--ink);font-weight:600}
.steps li.now .ic{background:var(--cherry);box-shadow:0 0 0 5px rgba(215,38,61,.15)}
.steps .th{font-family:var(--display);font-weight:500}
.steps .en{display:block;font-size:.82rem;color:var(--muted);font-weight:400}

/* ---------- หน้าร้าน ---------- */
.toolbar{display:flex;align-items:center;justify-content:space-between;gap:10px;margin:50px 2px
12px}
.toolbar .updated{color:var(--muted);font-size:.88rem}
.tabs{display:flex;gap:8px;overflow-x:auto;margin:0 -16px 14px;padding:2px 16px 4px;scrollbar-
width:none}
.tabs::-webkit-scrollbar{display:none}
.tab{flex:none;border:0;border-radius:999px;padding:10px 16px;background:#fff;color:var(--
leaf);font-size:.95rem;font-weight:500;cursor:pointer;box-shadow:inset 0 0 0 1.5px var(--line)}
.tab .n{font-family:var(--display);margin-left:7px;opacity:.75}
.tab.on{background:var(--leaf);color:#fff;box-shadow:none}

.order{position:relative;padding-left:26px;overflow:hidden}
.order::before{content:"";position:absolute;left:0;top:0;bottom:0;width:9px;background:var(--
stripe,#bbb)}
.order.s0{--stripe:#E9A23B}
.order.s1{--stripe:var(--cherry)}
.order.s2{--stripe:var(--ready)}
.order.s3{--stripe:#AAB8B0;opacity:.75}
.order .head{display:flex;align-items:center;justify-content:space-between;gap:10px}
.order .q{font-family:var(--display);font-size:1.9rem;line-height:1.1;color:var(--leaf)}
.order .badge{font-size:.85rem;font-weight:500;padding:4px 12px;border-
radius:999px;background:var(--field);color:var(--leaf)}
.order .items{margin:6px 0 0;font-weight:600;font-size:1.05rem}
.order .meta{display:flex;align-items:baseline;justify-content:space-between;gap:10px;margin:0 0
14px}
.order .who{margin:0;color:var(--muted);font-size:.93rem}
.order .amt{font-size:1.15rem;color:var(--leaf)}
.btn.act1{background:var(--cherry);box-shadow:0 12px 22px -12px rgba(215,38,61,.8)}
.btn.act2{background:var(--ready)}
.btn.act3{background:var(--leaf)}
.empty{text-align:center;color:var(--muted);padding:34px 14px}
.empty strong{display:block;font-family:var(--display);font-weight:500;font-
```

```
size:1.1rem;color:var(--leaf);margin-bottom:4px}

@media (prefers-reduced-motion:reduce){
  .ticket.print{animation:none}
  *{transition:none!important}
}
```

9.2 Backend (app/backend)

app/backend/Dockerfile

สร้าง image ของ Backend จาก Node.js 20 ติดตั้งแพ็กเกจแล้วรัน `server.js`

```
FROM node:20-alpine
WORKDIR /app
COPY package.json .
RUN npm install --omit=dev
COPY server.js .
EXPOSE 3000
CMD ["node", "server.js"]
```

app/backend/package.json

รายชื่อแพ็กเกจที่ใช้: `express` 5 (ทำ API) และ `pg` (ต่อ PostgreSQL)

```
{
  "name": "coffee-backend",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": { "start": "node server.js" },
  "dependencies": {
    "express": "^5.1.0",
    "pg": "^8.13.1"
  }
}
```

app/backend/server.js

หัวใจของ Backend: health check, API ทั้งหมด, บันทึกออร์เดอร์หลายรายการใน transaction เดียว, ตรวจ PIN, สร้าง 3 ตารางและเมนูตัวอย่าง และรอ DB พร้อมก่อน

```
// Backend API ของ Coffee Queue (Node.js + Express 5: error ใน async route ส่งไปที่ error handler ให้เอง)
// ค่าเชื่อมต่อ DB อ่านจาก environment variable: PGHOST, PGUSER, PGPASSWORD, PGDATABASE
// PIN ของพนักงานอ่านจาก STAFF_PIN
const express = require('express');
const os = require('os');
const { Pool } = require('pg');

const app = express();
const db = new Pool();
const PIN = process.env.STAFF_PIN || '1234';
const STATUSES = ['waiting', 'making', 'ready', 'done'];   // รอคิว -> กำลังทำ -> พร้อมรับ -> รับแล้ว
const MAX_QTY = 20;

app.use(express.json());
```

```
// ใส่ชื่อ pod/container ไว้ใน header ทุก response เพื่อดูว่า Load Balance กระจายจริง
app.use((req, res, next) => { res.set('X-Served-By', os.hostname()); next(); });

// ---------- ตัวช่วย ----------
const queueNo = (id) => 'A' + String(id).padStart(3, '0');           // 15 -> "A015"
const idOf = (q) => {                                                  // "A015" หรือ "15" -> 15
  const n = parseInt(String(q).replace(/\D/g, ''), 10);
  return Number.isInteger(n) && n > 0 ? n : null;
};

// อ่านออร์เดอร์พร้อมรายการเครื่องดื่ม แล้วจัดรูปแบบให้หน้าเว็บใช้ได้ทันที
async function loadOrders(whereSql, params) {
  const { rows } = await db.query(
    `SELECT o.id, o.customer_name, o.status, o.created_at,
            i.temp, i.quantity, m.name AS drink, m.price
       FROM orders o
       JOIN order_items i ON i.order_id = o.id
       JOIN menu_items m  ON m.id = i.menu_item_id
      ${whereSql}
      ORDER BY o.id, i.id`, params);
  const map = new Map();
  for (const r of rows) {
    if (!map.has(r.id)) {
      map.set(r.id, { id: r.id, queue: queueNo(r.id), name: r.customer_name,
                      status: r.status, created_at: r.created_at, items: [], total: 0 });
    }
    const o = map.get(r.id);
    o.items.push({ drink: r.drink, temp: r.temp, quantity: r.quantity });
    o.total += r.price * r.quantity;
  }
  return [...map.values()];
}

// ---------- Health check (ให้ Kubernetes ใช้) ----------
let dbReady = false;
app.get('/healthz', (req, res) => res.send('ok'));                     // แอปยังทำงานอยู่
app.get('/readyz', async (req, res) => {                               // ต่อ DB ได้ไหม
  if (!dbReady) return res.status(503).send('db not ready');
  try { await db.query('SELECT 1'); res.send('ready'); }
  catch { res.status(503).send('db not ready'); }
});

// ---------- API ฝั่งลูกค้า ----------
app.get('/api/whoami', (req, res) => res.json({ servedBy: os.hostname() }));

// เมนูทั้งหมด
app.get('/api/menu', async (req, res) => {
  const { rows } = await db.query('SELECT id, name, price FROM menu_items ORDER BY id');
  res.json(rows);
});

// สั่งเครื่องดื่ม: { name, items: [{ drink, temp, quantity }] } -> ได้เลขคิว
app.post('/api/orders', async (req, res) => {
  const name = String((req.body || {}).name || '').trim().slice(0, 40);
  const items = Array.isArray((req.body || {}).items) ? req.body.items : [];
  if (!name) return res.status(400).json({ error: 'Enter your name so the barista can call you.'
});
  if (!items.length) return res.status(400).json({ error: 'Pick at least one drink.' });

  const { rows: menu } = await db.query('SELECT id, name FROM menu_items');
```

```
  const byName = new Map(menu.map((m) => [m.name, m.id]));
  const clean = [];
  for (const it of items) {
    const menuId = byName.get(String(it.drink));
    if (!menuId) return res.status(400).json({ error: 'Unknown drink: ' + it.drink });
    clean.push({ menuId, temp: it.temp === 'iced' ? 'iced' : 'hot',
                 qty: Math.min(MAX_QTY, Math.max(1, parseInt(it.quantity, 10) || 1)) });
  }

  // บันทึกออร์เดอร์และทุกรายการใน transaction เดียว (สำเร็จทั้งหมดหรือไม่บันทึกเลย)
  const client = await db.connect();
  let orderId;
  try {
    await client.query('BEGIN');
    const r = await client.query('INSERT INTO orders (customer_name) VALUES ($1) RETURNING id',
[name]);
    orderId = r.rows[0].id;
    for (const c of clean) {
      await client.query(
        'INSERT INTO order_items (order_id, menu_item_id, temp, quantity) VALUES ($1, $2, $3,
$4)',
        [orderId, c.menuId, c.temp, c.qty]);
    }
    await client.query('COMMIT');
  } catch (e) {
    await client.query('ROLLBACK');
    throw e;
  } finally {
    client.release();
  }
  const [order] = await loadOrders('WHERE o.id = $1', [orderId]);
  res.status(201).json(order);
});

// ดูสถานะคิว: /api/orders/A015 หรือ /api/orders/15
app.get('/api/orders/:q', async (req, res) => {
  const id = idOf(req.params.q);
  const [order] = id ? await loadOrders('WHERE o.id = $1', [id]) : [];
  if (!order) return res.status(404).json({ error: 'Queue number not found' });
  res.json(order);
});

// ---------- API ฝั่งร้าน (ต้องส่ง PIN ใน header X-Staff-PIN) ----------
const staffOnly = (req, res, next) =>
  req.get('X-Staff-PIN') === PIN ? next() : res.status(401).json({ error: 'Wrong PIN. Please try
again.' });

// ออร์เดอร์ 100 รายการล่าสุด (หน้าร้านแยกแท็บตามสถานะเอง)
app.get('/api/staff/orders', staffOnly, async (req, res) => {
  res.json(await loadOrders('WHERE o.id IN (SELECT id FROM orders ORDER BY id DESC LIMIT 100)',
[]));
});

// เปลี่ยนสถานะ: { status: "making" }
app.patch('/api/orders/:q/status', staffOnly, async (req, res) => {
  const id = idOf(req.params.q);
  const status = (req.body || {}).status;
  if (!STATUSES.includes(status)) return res.status(400).json({ error: 'Invalid status' });
  const r = await db.query('UPDATE orders SET status = $1 WHERE id = $2', [status, id]);
  if (!r.rowCount) return res.status(404).json({ error: 'Queue number not found' });
  res.json({ ok: true, queue: queueNo(id), status });
```

```
});

// error ที่ไม่ได้คาดไว้ เช่น DB หลุดกลางคัน
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ error: 'Something went wrong. Please try again.' });
});

// ---------- สร้างตารางและเมนูตัวอย่างตอนเริ่ม ----------
async function initDb() {
  await db.query(`CREATE TABLE IF NOT EXISTS menu_items (
    id SERIAL PRIMARY KEY, name TEXT NOT NULL UNIQUE, price INT NOT NULL)`);
  await db.query(`CREATE TABLE IF NOT EXISTS orders (
    id SERIAL PRIMARY KEY, customer_name TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'waiting', created_at TIMESTAMP DEFAULT now())`);
  await db.query(`CREATE TABLE IF NOT EXISTS order_items (
    id SERIAL PRIMARY KEY,
    order_id INT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    menu_item_id INT NOT NULL REFERENCES menu_items(id),
    temp TEXT NOT NULL, quantity INT NOT NULL)`);
  const { rows } = await db.query('SELECT COUNT(*)::int AS n FROM menu_items');
  if (rows[0].n === 0) {
    await db.query(`INSERT INTO menu_items (name, price) VALUES
      ('Latte', 60), ('Americano', 50), ('Cappuccino', 60), ('Mocha', 65),
      ('Thai Tea', 45), ('Green Tea Latte', 55), ('Cocoa', 50), ('Chocolate', 55)`);
  }
}

// เปิดรับ request ทันที แล้วค่อยเตรียม DB อยู่เบื้องหลัง
// ระหว่างนี้ /healthz ตอบ ok แต่ /readyz ตอบ 503 จน DB พร้อม (Kubernetes จะยังไม่ส่ง traffic มา)
app.listen(3000, () => console.log('Backend พร้อมที่พอร์ต 3000'));
(async function waitForDb() {
  for (;;) {
    try { await initDb(); dbReady = true; console.log('Database พร้อมแล้ว'); break; }
    catch (e) { console.log('รอ database...', e.message); await new Promise((r) => setTimeout(r,
2000)); }
  }
})();
```

9.3 Docker Compose

app/docker-compose.yml

รัน 3 container ในเครื่องด้วยคำสั่งเดียว (อธิบายในส่วนที่ 4.2)

```
# รันทั้งระบบ: docker compose up --build  แล้วเปิด http://localhost:8080
services:
  frontend:                # Container 1: Nginx + หน้าเว็บ
    build: ./frontend
    ports:
      - "8080:80"          # เปิดออกมาที่ localhost:8080
    depends_on:
      - backend

  backend:                 # Container 2: Node.js API
    build: ./backend
    environment:
      PGHOST: db           # ชื่อ service ของ Database
      PGUSER: coffee
```

```
      PGPASSWORD: coffee123
      PGDATABASE: coffee
      STAFF_PIN: "1234"
    depends_on:
      - db

  db:                       # Container 3: PostgreSQL
    image: postgres:16
    environment:
      POSTGRES_USER: coffee
      POSTGRES_PASSWORD: coffee123
      POSTGRES_DB: coffee
    volumes:
      - dbdata:/var/lib/postgresql/data   # ข้อมูลไม่หายเมื่อลบ container

volumes:
  dbdata:
```

9.4 Kubernetes (k8s)

`k8s/00-namespace.yaml`

พื้นที่แยกของระบบ ทุก resource อยู่ใน namespace `coffee-queue`

```
# พื้นที่แยกของระบบเรา ทุก resource อยู่ใน namespace นี้
apiVersion: v1
kind: Namespace
metadata:
  name: coffee-queue
```

`k8s/01-config.yaml`

ค่าตั้งค่าที่ไม่ใช่ความลับ ส่งให้ Backend และ Postgres เป็น environment variable

```
# ค่าตั้งค่าทั่วไป (ไม่ใช่ความลับ) ส่งให้ Backend และ Postgres เป็น environment variable
# ส่วนรหัสผ่านอยู่ใน Secret ชื่อ coffee-secret ที่ deploy.yml สร้างให้ (ไม่เก็บใน Git)
apiVersion: v1
kind: ConfigMap
metadata:
  name: coffee-config
  namespace: coffee-queue
data:
  PGHOST: postgres          # ชื่อ Service ของ Database
  PGUSER: coffee
  PGDATABASE: coffee
```

`k8s/02-postgres.yaml`

Database แบบ StatefulSet ต่อดิสก์ถาวร (PVC 1Gi) และ Service แบบ ClusterIP ให้ Backend เรียกด้วยชื่อ `postgres`

```
# Database: PostgreSQL 1 ตัว แบบ StatefulSet เก็บข้อมูลใน PVC (ดิสก์ถาวร)
# ลบ pod แล้ว Kubernetes สร้างใหม่และต่อดิสก์เดิม ข้อมูลจึงไม่หาย
apiVersion: v1
kind: Service
metadata:
```

```yaml
  name: postgres
  namespace: coffee-queue
spec:
  type: ClusterIP           # ใช้ได้เฉพาะในระบบ ไม่เปิดออกข้างนอก
  selector:
    app: postgres
  ports:
    - port: 5432
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: coffee-queue
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_USER
              valueFrom: { configMapKeyRef: { name: coffee-config, key: PGUSER } }
            - name: POSTGRES_DB
              valueFrom: { configMapKeyRef: { name: coffee-config, key: PGDATABASE } }
            - name: POSTGRES_PASSWORD
              valueFrom: { secretKeyRef: { name: coffee-secret, key: PGPASSWORD } }
            - name: PGDATA
              value: /var/lib/postgresql/data/pgdata
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "coffee"]
            initialDelaySeconds: 5
            periodSeconds: 5
          resources:
            requests: { cpu: 100m, memory: 128Mi }
            limits: { cpu: 500m, memory: 512Mi }
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        storageClassName: local-path   # storage ที่มากับ k3s
        resources:
          requests:
            storage: 1Gi
```

k8s/03-backend.yaml

Backend 2 ตัว (Load Balance) มี liveness/readiness probe และ resource requests ที่ HPA ใช้คำนวณ

```yaml
# Backend: Node.js API รัน 2 ตัว (HPA เพิ่มได้ถึง 5) Service กระจาย request ให้ทุกตัว = Load Balance
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: coffee-queue
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: DOCKERHUB_USER/coffee-backend:APP_VERSION
          ports:
            - containerPort: 3000
          envFrom:
            - configMapRef:
                name: coffee-config
          env:
            - name: PGPASSWORD
              valueFrom: { secretKeyRef: { name: coffee-secret, key: PGPASSWORD } }
            - name: STAFF_PIN
              valueFrom: { secretKeyRef: { name: coffee-secret, key: STAFF_PIN } }
          resources:              # HPA ใช้ค่า requests คำนวณว่า CPU สูงแค่ไหน
            requests: { cpu: 100m, memory: 128Mi }
            limits: { cpu: 500m, memory: 256Mi }
          livenessProbe:          # แอปค้าง -> Kubernetes restart ให้
            httpGet: { path: /healthz, port: 3000 }
            initialDelaySeconds: 10
            periodSeconds: 10
          readinessProbe:         # DB ยังไม่พร้อม -> ยังไม่ส่ง traffic มา
            httpGet: { path: /readyz, port: 3000 }
            initialDelaySeconds: 5
            periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: backend               # ชื่อนี้ nginx.conf ใช้ (proxy_pass http://backend:3000)
  namespace: coffee-queue
spec:
  selector:
    app: backend
  ports:
    - port: 3000
```

k8s/04-frontend.yaml

Frontend 2 ตัว และ Service ชื่อ `frontend`

```yaml
# Frontend: Nginx + หน้าเว็บ รัน 2 ตัว
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: coffee-queue
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: DOCKERHUB_USER/coffee-frontend:APP_VERSION
          ports:
            - containerPort: 80
          resources:
            requests: { cpu: 50m, memory: 32Mi }
            limits: { cpu: 200m, memory: 128Mi }
          readinessProbe:
            httpGet: { path: /, port: 80 }
            periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: coffee-queue
spec:
  selector:
    app: frontend
  ports:
    - port: 80
```

`k8s/05-ingress.yaml`

Gateway: ทางเข้าเดียวจากข้างนอก แยก `/api` ไป Backend และ `/` ไป Frontend

```yaml
# Gateway: ทางเข้าเดียวจากข้างนอก (พอร์ต 80 ของทุกเครื่อง) ผ่าน Traefik ที่มากับ k3s
#   /api  -> backend
#   /     -> frontend
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: coffee-ingress
  namespace: coffee-queue
spec:
  ingressClassName: traefik
  rules:
    - http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
```

```
            service: { name: backend, port: { number: 3000 } }
          - path: /
            pathType: Prefix
            backend:
              service: { name: frontend, port: { number: 80 } }
```

`k8s/06-hpa.yaml`

Scaling: เพิ่ม Backend อัตโนมัติ 2–5 pod เมื่อ CPU เฉลี่ยเกิน 60%

```
# Scaling: ถ้า CPU เฉลี่ยของ Backend เกิน 60% ของ requests ให้เพิ่ม pod ได้ถึง 5 ตัว
# (k3s มี metrics-server มาให้แล้ว)
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend
  namespace: coffee-queue
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
  minReplicas: 2
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60
```

9.5 Ansible (ansible)

`ansible/ansible.cfg`

ตั้งค่า Ansible ให้ไม่ต้องตอบ yes ตอน SSH เครื่องใหม่

```
[defaults]
# ไม่ต้องตอบ yes ตอนต่อ SSH เครื่องใหม่ครั้งแรก (เครื่อง EC2 ของเราสร้างใหม่บ่อย)
host_key_checking = False
interpreter_python = /usr/bin/python3
timeout = 30
```

`ansible/requirements.yml`

collection ที่ต้องติดตั้ง (`amazon.aws` ใช้สร้าง EC2)

```
# ติดตั้งด้วย: ansible-galaxy collection install -r requirements.yml
collections:
  - name: amazon.aws
    version: ">=8.2.0"
```

`ansible/vars.yml` (ต้องแก้)

ค่าของบัญชี AWS, Key Pair, ขนาดเครื่อง และ Docker Hub ดูวิธีหาค่าในข้อ 8.4–8.5

```
# ===== แก้ค่าในไฟล์นี้ให้เป็นของตัวเองก่อนรัน (หาค่าได้แบบเดียวกับ Lab 2) =====
owner_id: "bXXXXXXXX"                  # รหัสนักศึกษาของเจ้าของบัญชี AWS
project_id: "DGT013130-COFFEE"         # ใช้ติด Tag ให้รู้ว่าเครื่องไหนเป็นของโปรเจกต์นี้

region: "ap-southeast-1"               # Singapore
vpc_id: "vpc-REPLACE"
subnet_id: "subnet-REPLACE"            # Subnet ที่ออกอินเทอร์เน็ตได้ (เหมือน Lab 2)
ami_id: "ami-REPLACE"                  # Ubuntu Server 24.04 LTS, 64-bit (x86)
key_name: "lab01-bXXXXXXXX-key"        # ชื่อ Key Pair บน AWS (ไม่มี .pem) สร้างใหม่ได้ตามคู่มือข้อ 3.5
ssh_key_file: "{{ lookup('ansible.builtin.env', 'HOME') }}/.ssh/lab2-key.pem"   # ไฟล์ .pem ของ
key_name ข้างบน
ssh_cidr: "{{ lookup('ansible.builtin.pipe', 'curl -4fsS https://checkip.amazonaws.com') | trim
}}/32"

server_type: "t3.small"                # k3s server ต้องการ RAM 2 GB
worker_type: "t3.small"
worker_count: 2                        # จำนวน worker (ปรับ 1 หรือ 2)

# image บน Docker Hub (ต้องตั้งเป็น Public)
dockerhub_user: "REPLACE"
app_version: "1.0.0"
```

`ansible/secrets.example.yml` (คัดลอกเป็น secrets.yml)

แม่แบบรหัส DB และ PIN ร้าน `secrets.yml` จะไม่ขึ้น Git

```
# คัดลอกไฟล์นี้เป็น secrets.yml แล้วตั้งรหัสเอง:  cp secrets.example.yml secrets.yml
# secrets.yml อยู่ใน .gitignore จะไม่ขึ้น Git
db_password: "change-me-please"
staff_pin: "1234"
```

`ansible/templates/inventory.ini.j2`

แม่แบบรายชื่อเครื่อง `provision.yml` ใช้สร้าง `inventory.ini` จาก IP จริง

```
[server]
server1 ansible_host={{ server_info.instances[0].public_ip_address }} private_ip={{
server_info.instances[0].private_ip_address }}

[workers]
{% for instance in worker_info.instances | sort(attribute='launch_time') %}
worker{{ loop.index }} ansible_host={{ instance.public_ip_address }} private_ip={{
instance.private_ip_address }}
{% endfor %}

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file={{ ssh_key_file }}
```

`ansible/provision.yml`

สร้าง Security Group และ EC2 (server 1 + worker ตาม `worker_count`) ด้วย `exact_count` จึงรันซ้ำได้ไม่สร้างเกิน แล้วเขียน `inventory.ini`

```
---
# ขั้นที่ 5.1: สร้าง Security Group และ EC2 (k3s server 1 + worker ตาม worker_count)
```

```yaml
# รันซ้ำได้: exact_count ตรวจเครื่องที่มีอยู่ก่อน จึงไม่สร้างเครื่องเกิน
- name: Provision EC2 for the Coffee Queue k3s cluster
  hosts: localhost
  connection: local
  gather_facts: false
  vars_files:
    - vars.yml

  tasks:
    - name: Check input before creating resources
      ansible.builtin.assert:
        that:
          - owner_id is match('^[A-Za-z0-9-]{5,32}$')
          - vpc_id is match('^vpc-[0-9a-f]+$')
          - subnet_id is match('^subnet-[0-9a-f]+$')
          - ami_id is match('^ami-[0-9a-f]+$')
          - (worker_count | int) in [1, 2]
          - ssh_cidr is match('^[0-9.]+/32$')
        fail_msg: "Edit vars.yml with your real values before running this playbook."

    - name: Create the cluster security group
      amazon.aws.ec2_security_group:
        name: "coffee-{{ owner_id }}-sg"
        description: "Coffee Queue k3s cluster"
        region: "{{ region }}"
        vpc_id: "{{ vpc_id }}"
        rules:
          - proto: tcp                    # SSH จากเครื่องเราเท่านั้น
            from_port: 22
            to_port: 22
            cidr_ip: "{{ ssh_cidr }}"
          - proto: tcp                    # ผู้ใช้เปิดเว็บผ่าน Ingress
            from_port: 80
            to_port: 80
            cidr_ip: "0.0.0.0/0"
          - proto: all                    # เครื่องใน cluster คุยกันได้ทุกพอร์ต (6443, 8472, 10250)
            group_name: "coffee-{{ owner_id }}-sg"
        tags:
          Owner: "{{ owner_id }}"
          Project: "{{ project_id }}"
          Role: "cluster-sg"
      register: cluster_sg

    - name: Ensure one k3s server exists
      amazon.aws.ec2_instance:
        name: "coffee-{{ owner_id }}-server"
        region: "{{ region }}"
        image_id: "{{ ami_id }}"
        instance_type: "{{ server_type }}"
        key_name: "{{ key_name }}"
        network_interfaces:
          - subnet_id: "{{ subnet_id }}"
            assign_public_ip: true
            groups:
              - "{{ cluster_sg.group_id }}"
        volumes:
          - device_name: /dev/sda1
            ebs:
              volume_size: 20
              volume_type: gp3
              delete_on_termination: true
```

```
      exact_count: 1
      filters:
        "tag:Owner": "{{ owner_id }}"
        "tag:Project": "{{ project_id }}"
        "tag:Role": "server"
        "vpc-id": "{{ vpc_id }}"
        "instance-state-name": [pending, running, stopping, stopped]
      tags:
        Owner: "{{ owner_id }}"
        Project: "{{ project_id }}"
        Role: "server"
      state: started
      wait: true

  - name: Ensure the k3s workers exist
    amazon.aws.ec2_instance:
      name: "coffee-{{ owner_id }}-worker"
      region: "{{ region }}"
      image_id: "{{ ami_id }}"
      instance_type: "{{ worker_type }}"
      key_name: "{{ key_name }}"
      network_interfaces:
        - subnet_id: "{{ subnet_id }}"
          assign_public_ip: true
          groups:
            - "{{ cluster_sg.group_id }}"
      volumes:
        - device_name: /dev/sda1
          ebs:
            volume_size: 20
            volume_type: gp3
            delete_on_termination: true
      exact_count: "{{ worker_count | int }}"
      filters:
        "tag:Owner": "{{ owner_id }}"
        "tag:Project": "{{ project_id }}"
        "tag:Role": "worker"
        "vpc-id": "{{ vpc_id }}"
        "instance-state-name": [pending, running, stopping, stopped]
      tags:
        Owner: "{{ owner_id }}"
        Project: "{{ project_id }}"
        Role: "worker"
      state: started
      wait: true

  - name: Read the running server
    amazon.aws.ec2_instance_info:
      region: "{{ region }}"
      filters:
        "tag:Owner": "{{ owner_id }}"
        "tag:Project": "{{ project_id }}"
        "tag:Role": "server"
        "vpc-id": "{{ vpc_id }}"
        "instance-state-name": running
    register: server_info

  - name: Read the running workers
    amazon.aws.ec2_instance_info:
      region: "{{ region }}"
      filters:
```

```
          "tag:Owner": "{{ owner_id }}"
          "tag:Project": "{{ project_id }}"
          "tag:Role": "worker"
          "vpc-id": "{{ vpc_id }}"
          "instance-state-name": running
      register: worker_info

    - name: Confirm the instance counts
      ansible.builtin.assert:
        that:
          - server_info.instances | length == 1
          - worker_info.instances | length == (worker_count | int)

    - name: Write the inventory from the running instances
      ansible.builtin.template:
        src: templates/inventory.ini.j2
        dest: "{{ playbook_dir }}/inventory.ini"
        mode: "0644"

    - name: Wait until SSH is ready on every instance
      ansible.builtin.wait_for:
        host: "{{ item.public_ip_address }}"
        port: 22
        timeout: 300
      loop: "{{ server_info.instances + worker_info.instances }}"
      loop_control:
        label: "{{ item.instance_id }}"

    - name: Show instance IDs for the record
      ansible.builtin.debug:
        msg:
          server_id: "{{ server_info.instances[0].instance_id }}"
          worker_ids: "{{ worker_info.instances | map(attribute='instance_id') | list }}"
          server_public_ip: "{{ server_info.instances[0].public_ip_address }}"
```

ansible/k3s.yml

ติดตั้ง k3s บน server อ่าน token แล้วให้ worker join จากนั้นรอจนทุก node เป็น Ready

```
---
# ขั้นที่ 5.2: ติดตั้ง k3s บน server แล้วให้ worker เข้าร่วม cluster
# รันซ้ำได้: ถ้าติดตั้งแล้ว (มีไฟล์ k3s อยู่) จะข้ามไป
- name: Install the k3s server
  hosts: server
  become: true
  gather_facts: false
  tasks:
    - name: Install k3s server
      ansible.builtin.shell: >
        curl -sfL https://get.k3s.io |
        INSTALL_K3S_EXEC="server --write-kubeconfig-mode 644 --node-external-ip {{ ansible_host
}}" sh -
      args:
        creates: /usr/local/bin/k3s

    - name: Wait for the join token
      ansible.builtin.wait_for:
        path: /var/lib/rancher/k3s/server/node-token
        timeout: 120
```

```
    - name: Read the join token
      ansible.builtin.slurp:
        src: /var/lib/rancher/k3s/server/node-token
      register: token_file

    - name: Keep the token for the workers
      ansible.builtin.set_fact:
        k3s_token: "{{ token_file.content | b64decode | trim }}"

- name: Join the workers to the cluster
  hosts: workers
  become: true
  gather_facts: false
  vars:
    server_ip: "{{ hostvars['server1'].private_ip }}"
    k3s_token: "{{ hostvars['server1'].k3s_token }}"
  tasks:
    - name: Install k3s agent
      ansible.builtin.shell: >
        curl -sfL https://get.k3s.io |
        K3S_URL="https://{{ server_ip }}:6443" K3S_TOKEN="{{ k3s_token }}" sh -
      args:
        creates: /usr/local/bin/k3s-agent-uninstall.sh
      no_log: true

- name: Check that every node is Ready
  hosts: server
  become: true
  gather_facts: false
  tasks:
    - name: Wait until all nodes are Ready
      ansible.builtin.shell: k3s kubectl get nodes --no-headers | awk '$2=="Ready"' | wc -l
      register: ready_nodes
      until: ready_nodes.stdout | int == (groups['workers'] | length + 1)
      retries: 30
      delay: 10
      changed_when: false

    - name: Show the nodes
      ansible.builtin.command: k3s kubectl get nodes -o wide
      register: nodes
      changed_when: false

    - name: Print nodes
      ansible.builtin.debug:
        var: nodes.stdout_lines
```

`ansible/deploy.yml`

คัดลอกไฟล์ `k8s/` ไปที่ server ใส่ชื่อ image สร้าง Secret แล้ว `kubectl apply` และรอจนทุก pod พร้อม

```
---
# ขั้นที่ 5.3: deploy แอปขึ้น cluster (รันซ้ำได้ ใช้ตอนอัปเดตเวอร์ชันด้วย)
- name: Deploy Coffee Queue to k3s
  hosts: server
  become: true
  gather_facts: false
  vars_files:
    - vars.yml
    - secrets.yml
```

```
  vars:
    k8s_dir: /opt/coffee-queue/k8s

  tasks:
    - name: Copy the Kubernetes files to the server
      ansible.builtin.copy:
        src: ../k8s/
        dest: "{{ k8s_dir }}/"
        mode: "0644"

    - name: Put the Docker Hub user and version into the image names
      ansible.builtin.replace:
        path: "{{ k8s_dir }}/{{ item }}"
        regexp: 'DOCKERHUB_USER/(coffee-[a-z]+):APP_VERSION'
        replace: '{{ dockerhub_user }}/\1:{{ app_version }}'
      loop:
        - 03-backend.yaml
        - 04-frontend.yaml

    - name: Create the namespace
      ansible.builtin.command: k3s kubectl apply -f {{ k8s_dir }}/00-namespace.yaml
      changed_when: false

    - name: Create or update the Secret (DB password and staff PIN)
      ansible.builtin.shell: >
        k3s kubectl -n coffee-queue create secret generic coffee-secret
        --from-literal=PGPASSWORD='{{ db_password }}'
        --from-literal=STAFF_PIN='{{ staff_pin }}'
        --dry-run=client -o yaml | k3s kubectl apply -f -
      no_log: true
      changed_when: false

    - name: Apply all Kubernetes files
      ansible.builtin.command: k3s kubectl apply -f {{ k8s_dir }}/
      register: applied
      changed_when: "'configured' in applied.stdout or 'created' in applied.stdout"

    - name: Wait for the database
      ansible.builtin.command: k3s kubectl -n coffee-queue rollout status statefulset/postgres --
timeout=300s
      changed_when: false

    - name: Wait for the backend and frontend
      ansible.builtin.command: k3s kubectl -n coffee-queue rollout status deployment/{{ item }} -
-timeout=300s
      loop: [backend, frontend]
      changed_when: false

    - name: Show where everything runs
      ansible.builtin.command: k3s kubectl -n coffee-queue get pods,svc,ingress,hpa -o wide
      register: status
      changed_when: false

    - name: Print status
      ansible.builtin.debug:
        var: status.stdout_lines

    - name: Open the website
      ansible.builtin.debug:
```

```
        msg: "Open http://{{ ansible_host }}/  (staff page: http://{{ ansible_host
}}/staff.html)"
```

`ansible/destroy.yml`

ค้นเครื่องจาก Tag `Owner` + `Project` แล้ว terminate และลบ Security Group (ไม่แตะเครื่องของ Lab อื่น)

```yaml
---
# ขั้นที่ 7: ลบเฉพาะทรัพยากรของโปรเจกต์นี้ (ค้นจาก Tag Owner + Project)
- name: Remove the Coffee Queue AWS resources
  hosts: localhost
  connection: local
  gather_facts: false
  vars_files:
    - vars.yml

  tasks:
    - name: Check identifiers
      ansible.builtin.assert:
        that:
          - owner_id is match('^[A-Za-z0-9-]{5,32}$')
          - vpc_id is match('^vpc-[0-9a-f]+$')

    - name: Find this project's EC2 instances
      amazon.aws.ec2_instance_info:
        region: "{{ region }}"
        filters:
          "tag:Owner": "{{ owner_id }}"
          "tag:Project": "{{ project_id }}"
          "tag:Role": [server, worker]
          "vpc-id": "{{ vpc_id }}"
          "instance-state-name": [pending, running, stopping, stopped]
      register: found

    - name: Show the instance IDs to remove
      ansible.builtin.debug:
        msg: "{{ found.instances | map(attribute='instance_id') | list }}"

    - name: Terminate the instances
      amazon.aws.ec2_instance:
        region: "{{ region }}"
        instance_ids: "{{ found.instances | map(attribute='instance_id') | list }}"
        state: absent
        wait: true
      when: found.instances | length > 0

    - name: Delete the cluster security group
      amazon.aws.ec2_security_group:
        region: "{{ region }}"
        name: "coffee-{{ owner_id }}-sg"
        vpc_id: "{{ vpc_id }}"
        state: absent

    - name: Remove the old inventory file
      ansible.builtin.file:
        path: "{{ playbook_dir }}/inventory.ini"
        state: absent
```

9.6 Scripts และ Load test

scripts/build-push.sh

build image แบบ x86 แล้ว push ขึ้น Docker Hub

```
#!/usr/bin/env bash
# build image ของ frontend และ backend แล้ว push ขึ้น Docker Hub
# ใช้: ./scripts/build-push.sh <dockerhub_user> <version>
# ตัวอย่าง: ./scripts/build-push.sh boss123 1.0.0
# --platform linux/amd64 ทำให้ image รันบน EC2 (x86) ได้ แม้จะ build บน Mac M1/M2
set -euo pipefail
USER_NAME="${1:?ใส่ชื่อผู้ใช้ Docker Hub}"
VERSION="${2:-1.0.0}"
cd "$(dirname "$0")/../app"

docker buildx build --platform linux/amd64 -t "$USER_NAME/coffee-frontend:$VERSION" --push
./frontend
docker buildx build --platform linux/amd64 -t "$USER_NAME/coffee-backend:$VERSION"  --push
./backend

echo "เสร็จแล้ว: $USER_NAME/coffee-frontend:$VERSION และ $USER_NAME/coffee-backend:$VERSION"
```

scripts/check-lb.sh

ยิง 10 request แล้วดู header `X-Served-By` ว่า pod ไหนตอบ

```
#!/usr/bin/env bash
# ยิง request 10 ครั้ง แล้วดูว่า backend pod ไหนตอบ (header X-Served-By)
# ใช้: ./scripts/check-lb.sh <Public IP ของเครื่องใดก็ได้ใน cluster>
set -euo pipefail
IP="${1:?ใส่ Public IP}"
for i in $(seq 1 10); do
  curl -s -o /dev/null -D - "http://$IP/api/menu" | grep -i '^x-served-by'
done
```

load-test/order-load.js

k6 จำลองช่วงพักเที่ยง: เพิ่มเป็น 50 คนใน 1 นาที ค้างไว้ 3 นาที แล้วลดลง แต่ละออร์เดอร์มี 1–2 รายการ

```
// จำลองช่วงพักเที่ยง: คนเปิดเมนูและสั่งเครื่องดื่มพร้อมกันจำนวนมาก
// ใช้: k6 run -e TARGET=http://<Public IP> load-test/order-load.js
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '1m', target: 50 },   // ค่อยๆ เพิ่มเป็น 50 คน
    { duration: '3m', target: 50 },   // ค้างไว้ 3 นาที ให้ HPA มีเวลาเพิ่ม pod
    { duration: '1m', target: 0 },    // ลดลง
  ],
};

const BASE = __ENV.TARGET || 'http://localhost:8080';
const NAMES = ['Boss', 'Fon', 'Mew', 'Ploy', 'Nan', 'Tee'];
const DRINKS = ['Latte', 'Americano', 'Cappuccino', 'Mocha', 'Thai Tea', 'Green Tea Latte',
'Cocoa', 'Chocolate'];
```

```
const pick = (arr) => arr[Math.floor(Math.random() * arr.length)];

export default function () {
  const menu = http.get(`${BASE}/api/menu`);
  check(menu, { 'menu ok': r => r.status === 200 });

  // หนึ่งออร์เดอร์มี 1-2 รายการ เหมือนลูกค้าจริง
  const items = [{ drink: pick(DRINKS), temp: Math.random() < 0.5 ? 'hot' : 'iced', quantity: 1
}];
  if (Math.random() < 0.5) items.push({ drink: pick(DRINKS), temp: 'iced', quantity: 1 });
  const body = JSON.stringify({ name: pick(NAMES), items });
  const order = http.post(`${BASE}/api/orders`, body, { headers: { 'Content-Type':
'application/json' } });
  check(order, { 'order created': r => r.status === 201 });

  sleep(0.5);
}
```

`.gitignore`

ไฟล์ที่ห้ามขึ้น Git: รหัสผ่าน, inventory และกุญแจ `.pem`

```
node_modules/
ansible/inventory.ini
ansible/secrets.yml
*.pem
```